

In [140...

```
import numpy as np
import pandas as pd

import statsmodels.api as sm
import scipy.stats as stats

from sklearn.linear_model import LinearRegression, LassoCV
from sklearn.preprocessing import OneHotEncoder, StandardScaler, PolynomialF
from sklearn.feature_selection import RFE
from sklearn.impute import SimpleImputer
from sklearn.model_selection import LeaveOneOut, cross_val_score, KFold
from sklearn.metrics import mean_absolute_error, r2_score

import matplotlib.pyplot as plt
import seaborn as sns
```

Task 1 Linear Regression

In this assignment we will deal with tools and methods for visualizing data and computing some simple characteristic measures. Our aim here is to apply all the basic techniques and to draw correct conclusions. The data we work with is the Ames Housing data set available for download at Kaggle:

[Kaggle - Ames Housing Dataset](#)

or included into the AmesHousing package in R. The full data set contains almost 3000 observations on 81 features. The overall objective is to explain the sale prices SalePrice.

Task 1.1

Before fitting a model to the data, first we have a closer look at the definitions of the variables and decide if some of them might require a separate treatment. Consider, for example, the first variable MSSubClass. It takes textual values and obviously cannot be included into the linear model as it is. There are two approaches how this can be handled. One approach is to assign numbers from 1 to 16 to each possible outcome. Another approach is to create new dummy variables that are equal one for a specific outcome and zero else. Discuss these two approaches and suggest which approach is more appropriate here. Motivate your decision and keep in mind the nominal, ordinal, and cardinal scales from Statistics. Run a simple regression of SalePrices on MSSubClass taken as it is (without creating dummies or 1,...,16 variable). Check how does your software handle this type of a variable. (In R your data must be stored as a data frame.)

```
In [3]: data = pd.read_csv('/content/train.csv')
data.head()
```

```
Out[3]:
```

	Id	MSSubClass	MSZoning	LotFrontage	LotArea	Street	Alley	LotShape
0	1	60	RL	65.0	8450	Pave	NaN	Reg
1	2	20	RL	80.0	9600	Pave	NaN	Reg
2	3	60	RL	68.0	11250	Pave	NaN	IR1
3	4	70	RL	60.0	9550	Pave	NaN	IR1
4	5	60	RL	84.0	14260	Pave	NaN	IR1

5 rows × 81 columns

```
In [4]: print("Dataset shape:", data.shape)
print("Columns with missing values:\n", data.isnull().sum()[data.isnull().su

print("\nSalePrice statistics:")
print(data['SalePrice'].describe())

print("\nMSSubClass unique values and counts:")
print(data['MSSubClass'].value_counts())
```

```
Dataset shape: (1460, 81)
Columns with missing values:
  LotFrontage      259
Alley             1369
MasVnrType        872
MasVnrArea         8
BsmtQual          37
BsmtCond          37
BsmtExposure      38
BsmtFinType1      37
BsmtFinType2      38
Electrical         1
FireplaceQu       690
GarageType         81
GarageYrBlt        81
GarageFinish       81
GarageQual         81
GarageCond         81
PoolQC            1453
Fence             1179
MiscFeature       1406
dtype: int64
```

```
SalePrice statistics:
count      1460.000000
mean      180921.195890
std        79442.502883
min        34900.000000
25%       129975.000000
50%       163000.000000
75%       214000.000000
max        755000.000000
Name: SalePrice, dtype: float64
```

```
MSSubClass unique values and counts:
MSSubClass
20      536
60      299
50      144
120      87
30       69
160      63
70       60
80       58
90       52
190      30
85       20
75       16
45       12
180      10
40        4
Name: count, dtype: int64
```

```
In [21]: # Regression with MSSubClass as numeric
data['MSSubClass_numeric'] = pd.Categorical(data['MSSubClass']).codes
X_numeric = sm.add_constant(data['MSSubClass_numeric'])
```

```

y = data['SalePrice']
model_numeric = sm.OLS(y, X_numeric).fit()
print("\nRegression results with MSSubClass as numeric:")
print(model_numeric.summary())

```

Regression results with MSSubClass as numeric:

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice  R-squared:                0.0
03
Model:                  OLS      Adj. R-squared:            0.0
02
Method:                 Least Squares  F-statistic:              4.3
91
Date:                   Fri, 20 Dec 2024  Prob (F-statistic):      0.03
63
Time:                   00:09:04    Log-Likelihood:           -1854
2.
No. Observations:      1460      AIC:                     3.709e+
04
Df Residuals:          1458      BIC:                     3.710e+
04
Df Model:               1
Covariance Type:       nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
0.975]					

const	1.733e+05	4193.704	41.321	0.000	1.65e+05
1.82e+05					
MSSubClass_numeric	1228.5232	586.288	2.095	0.036	78.465
2378.581					

```

=====
==
Omnibus:                608.599  Durbin-Watson:            2.0
06
Prob(Omnibus):          0.000    Jarque-Bera (JB):          3381.1
50
Skew:                   1.878    Prob(JB):                  0.
00
Kurtosis:               9.440    Cond. No.                  1
4.7
=====
=====

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

```

In [33]: # Regression with MSSubClass as categorical
data['MSSubClass'] = data['MSSubClass'].astype(str)
X_dummies = pd.get_dummies(data['MSSubClass'], drop_first=True)
X_dummies = sm.add_constant(X_dummies)

```

```
X_dummies = X_dummies.astype(float)
y = data['SalePrice']
model_dummies = sm.OLS(y, X_dummies).fit()
print("\nRegression results with MSSubClass as categorical:")
print(model_dummies.summary())
```

Regression results with MSSubClass as categorical:

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice  R-squared:                0.2
46
Model:                  OLS      Adj. R-squared:              0.2
39
Method:                 Least Squares  F-statistic:            33.
73
Date:                   Fri, 20 Dec 2024  Prob (F-statistic):      8.66e-
79
Time:                   00:17:52    Log-Likelihood:           -1833
8.
No. Observations:      1460      AIC:                    3.671e+
04
Df Residuals:          1445      BIC:                    3.678e+
04
Df Model:               14
Covariance Type:       nonrobust
=====
```

```
=====
==
               coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const      2.008e+05    7429.878     27.023     0.000     1.86e+05     2.15e+
05
160      -6.213e+04     1.15e+04     -5.419     0.000    -8.46e+04    -3.96e+
04
180      -9.848e+04     2.31e+04     -4.256     0.000    -1.44e+05    -5.31e+
04
190      -7.117e+04     1.47e+04     -4.850     0.000    -9.99e+04    -4.24e+
04
20      -1.555e+04     8010.199     -1.942     0.052    -3.13e+04     158.5
94
30      -1.049e+05     1.12e+04     -9.394     0.000    -1.27e+05    -8.3e+
04
40      -4.465e+04     3.54e+04     -1.260     0.208    -1.14e+05     2.49e+
04
45      -9.219e+04     2.13e+04     -4.320     0.000    -1.34e+05    -5.03e+
04
50      -5.748e+04     9410.364     -6.108     0.000    -7.59e+04    -3.9e+
04
60       3.917e+04     8441.891      4.640     0.000     2.26e+04     5.57e+
04
70      -3.401e+04     1.16e+04     -2.924     0.004    -5.68e+04    -1.12e+
04
75      -8341.5805     1.89e+04     -0.442     0.658    -4.53e+04     2.86e+
04
80      -3.104e+04     1.17e+04     -2.642     0.008    -5.41e+04    -7998.2
20
85      -5.297e+04     1.72e+04     -3.082     0.002    -8.67e+04    -1.93e+
04
90      -6.724e+04     1.21e+04     -5.535     0.000    -9.11e+04    -4.34e+
04
```

```
=====
==
Omnibus:                    708.394    Durbin-Watson:                1.9
98
Prob(Omnibus):              0.000    Jarque-Bera (JB):            5221.1
98
Skew:                      2.138    Prob(JB):                    0.
00
Kurtosis:                  11.218    Cond. No.                    2
2.4
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

The first approach treats `MSSubClass` as a numeric variable, assuming a linear relationship between its values and `SalePrice`. This results in a very low R-squared value of 0.003, indicating that the model explains almost none of the variance in `SalePrice`. While the regression coefficient for `MSSubClass_numeric` is statistically significant (p-value < 0.05), this approach assumes an ordinal or cardinal scale, which is inappropriate since `MSSubClass` represents discrete, unordered categories.

The second approach treats `MSSubClass` as a categorical variable by creating dummy variables for each category. This allows the model to capture the unique impact of each category on `SalePrice`, leading to a significantly higher R-squared value of 0.246. Many of the coefficients for the dummy variables are statistically significant, indicating distinct impacts of different `MSSubClass` categories on the target variable. Given that `MSSubClass` is a nominal variable, the dummy variable approach is more appropriate, as it respects the unordered nature of the categories.

Task 1.2

Consider the following numeric explanatory variables `LotFrontage`, `LotArea`, `YearBuilt`, `BsmtFinSF1`, `BsmtUnfSF`. Plot the scatter plot of `SalePrices` vs. these variables. Analyse every plot and discuss the potential problems, transformations, or extensions of the model to take the patterns into account. Remember that you can either transform variables or create new variables to capture non-standard patterns.

```
In [36]: variables = ['LotFrontage', 'LotArea', 'YearBuilt', 'BsmtFinSF1', 'BsmtUnfSF']

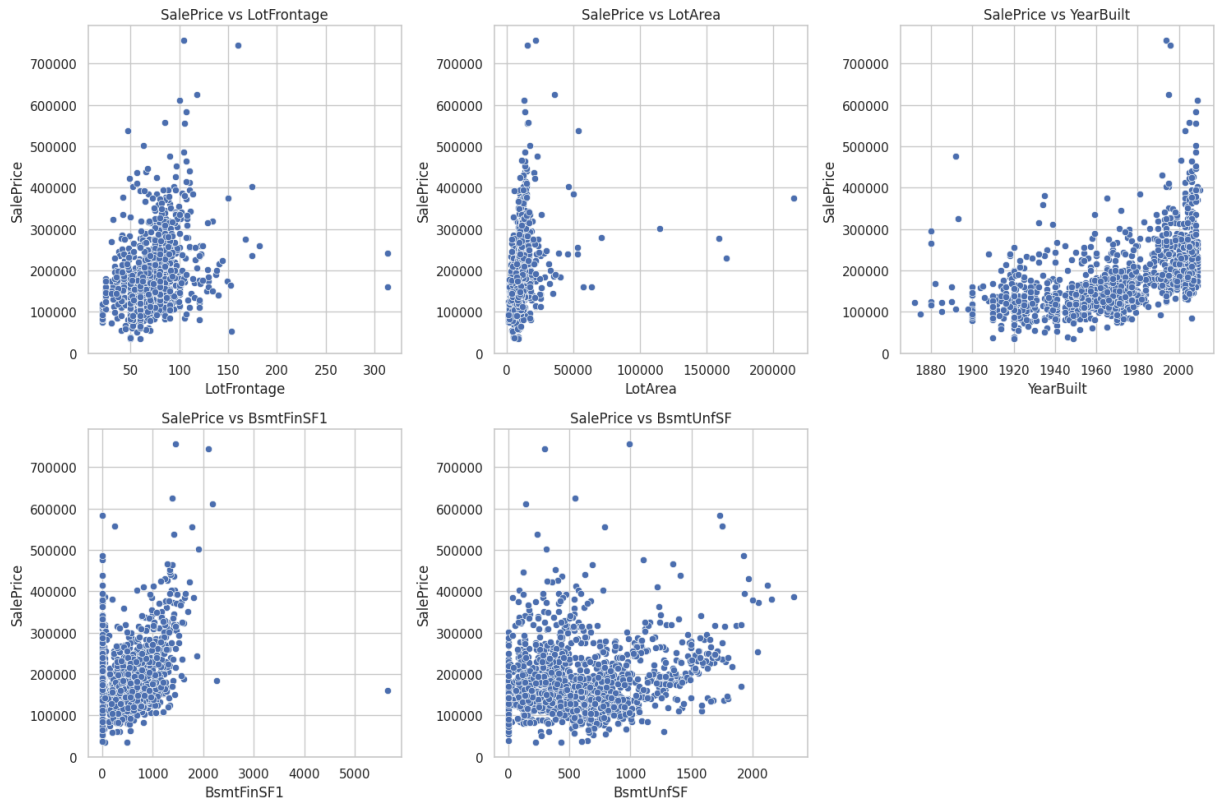
sns.set(style="whitegrid")
plt.figure(figsize=(15, 10))
```

```

for i, var in enumerate(variables, 1):
    plt.subplot(2, 3, i)
    sns.scatterplot(x=data[var], y=data['SalePrice'])
    plt.title(f'SalePrice vs {var}')
    plt.xlabel(var)
    plt.ylabel('SalePrice')

plt.tight_layout()
plt.show()

```



- LotFrontage vs. SalePrice: there is a weak positive relationship, but the pattern is non-linear, moreover several outliers with extremely high LotFrontage values distort the relationship.
- LotArea vs. SalePrice: there is a weak relationship, with a few very large LotArea values acting as outliers, while most data points are clustered at lower values of LotArea .
- YearBuilt vs. SalePrice: there is a clear positive trend, indicating newer houses tend to have higher sale prices.
- BsmtFinSF1 vs. SalePrice: an non-linear positive relationship containing some outliers with high BsmtFinSF1 but moderate SalePrice .
- BsmtUnfSF vs. SalePrice: the relationship is weak and scattered, with no clear trend.

Applying log transformations to LotFrontage , LotArea , BsmtFinSF1 , and BsmtUnfSF can normalize their distributions and reduce the impact of outliers. We can also create an interaction terms between YearBuilt and other

variables can capture the combined effects of house age and size on price. To simplify the non-linear trend of `YearBuilt`, we can apply binning into categories like decades. Lastly, combining `BsmtFinSF1` and `BsmtUnfSF` into a single feature, such as `TotalBsmtSF`, can provide a more robust measure.

```
In [38]: # Log transformations to reduce skewness
data['LotFrontage_log'] = np.log1p(data['LotFrontage'])
data['LotArea_log'] = np.log1p(data['LotArea'])
data['BsmtFinSF1_log'] = np.log1p(data['BsmtFinSF1'])
data['BsmtUnfSF_log'] = np.log1p(data['BsmtUnfSF'])

# Total Basement Square Feet feature
data['TotalBsmtSF'] = data['BsmtFinSF1'] + data['BsmtUnfSF']

# Binning YearBuilt into decades
data['YearBuilt_Binned'] = pd.cut(data['YearBuilt'],
                                   bins=[1870, 1900, 1920, 1940, 1960, 1980,
                                           2000],
                                   labels=['<1900', '1900-1920', '1920-1940',
                                           '1940-1960', '1960-1980', '1980-2000', '2000+'])

# Interaction term: TotalBsmtSF and YearBuilt
data['Bsmt_YearBuilt_Interaction'] = data['TotalBsmtSF'] * data['YearBuilt']

print(data[['LotFrontage_log', 'LotArea_log', 'BsmtFinSF1_log',
             'BsmtUnfSF_log', 'TotalBsmtSF', 'YearBuilt_Binned',
             'Bsmt_YearBuilt_Interaction']].head())
```

	LotFrontage_log	LotArea_log	BsmtFinSF1_log	BsmtUnfSF_log	TotalBsmtSF
0	4.189655	9.042040	6.561031	5.017280	856
1	4.394449	9.169623	6.886532	5.652489	1262
2	4.234107	9.328212	6.188264	6.075346	920
3	4.110874	9.164401	5.379897	6.293419	756
4	4.442651	9.565284	6.486161	6.196444	1145

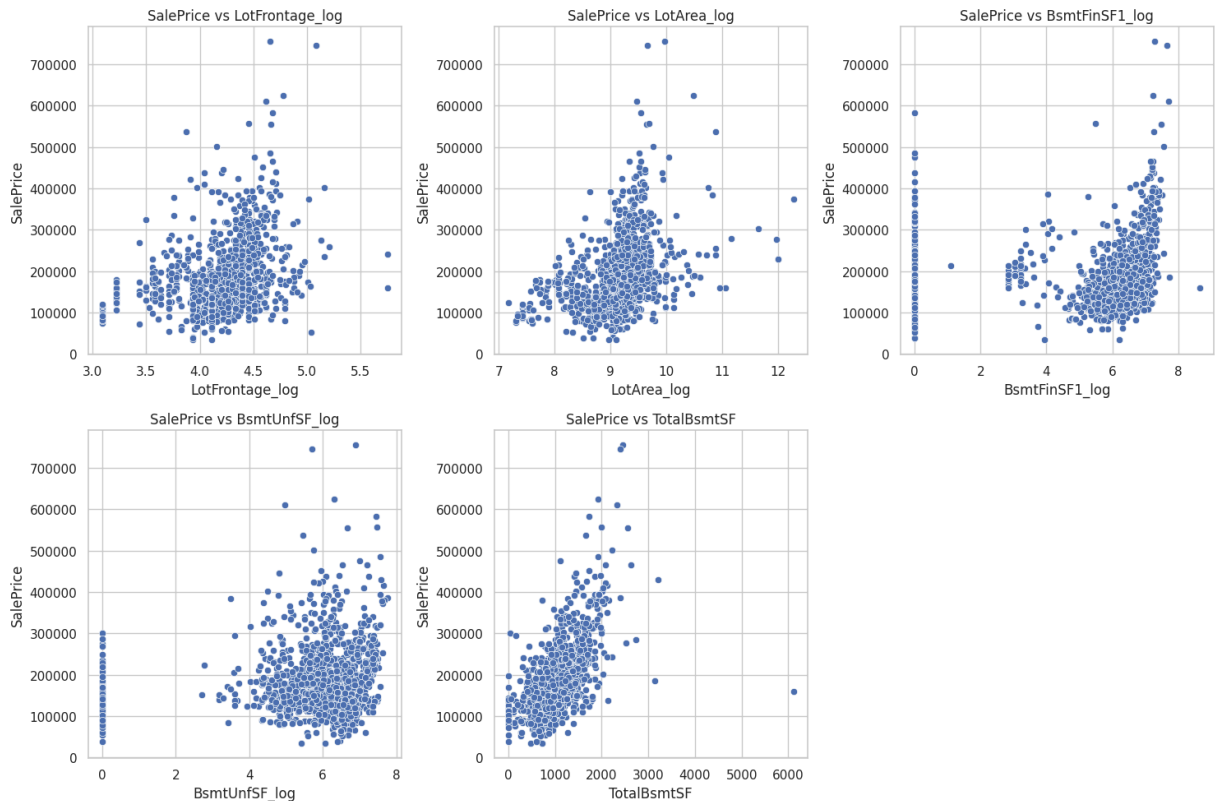
	YearBuilt_Binned	Bsmt_YearBuilt_Interaction
0	2000+	1714568
1	1960-1980	2493712
2	2000+	1840920
3	1900-1920	1447740
4	1980-2000	2290000

```
In [39]: transformed_variables = ['LotFrontage_log', 'LotArea_log', 'BsmtFinSF1_log',
                                   'BsmtUnfSF_log', 'TotalBsmtSF', 'YearBuilt_Binned',
                                   'Bsmt_YearBuilt_Interaction']

sns.set(style="whitegrid")
plt.figure(figsize=(15, 10))

for i, var in enumerate(transformed_variables, 1):
    plt.subplot(2, 3, i)
    sns.scatterplot(x=data[var], y=data['SalePrice'])
    plt.title(f'SalePrice vs {var}')
    plt.xlabel(var)
    plt.ylabel('SalePrice')
```

```
plt.tight_layout()
plt.show()
```



The log transformations have helped normalize the variables and reduce the effect of outliers. For `LotFrontage_log`, the relationship with `SalePrice` is still weak, and there is a lot of scatter, meaning it might not be a strong predictor on its own. Similarly, `BsmtUnfSF_log` doesn't show a clear trend and seems to have limited impact on `SalePrice`.

On the other hand, `LotArea_log` and `BsmtFinSF1_log` show stronger positive relationships with `SalePrice`, with higher values generally corresponding to higher prices. Combining the basement variables into `TotalBsmtSF` was effective, as it shows a clear positive trend with `SalePrice`, making it a useful feature for prediction too.

Task 1.3

Fit a simple linear regression without making the transformations discussed above. Pick up one of the numeric regressors, for example, `LotArea`. Write down the corresponding hypothesis of the t-test. Provide the formula for the test statistics, explain the components of the formula, and give the values for these components. Assess the goodness-of-fit of the model. Explain in your own words the difference between R^2 and adjusted R^2 .

```
In [40]: X = data['LotArea']  
y = data['SalePrice']  
  
X = sm.add_constant(X)  
model = sm.OLS(y, X).fit()  
  
print(model.summary())
```

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice    R-squared:                0.0
70
Model:                  OLS          Adj. R-squared:            0.0
69
Method:                 Least Squares    F-statistic:              10
9.1
Date:                  Fri, 20 Dec 2024    Prob (F-statistic):       1.12e-
24
Time:                  00:46:00          Log-Likelihood:           -1849
1.
No. Observations:      1460            AIC:                     3.699e+
04
Df Residuals:          1458            BIC:                     3.700e+
04
Df Model:               1
Covariance Type:       nonrobust
=====

```

```

=====
==
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const      1.588e+05    2914.717     54.495     0.000     1.53e+05    1.65e+
05
LotArea      2.1000      0.201     10.445     0.000      1.706      2.4
94
=====

```

```

=====
==
Omnibus:              587.660    Durbin-Watson:              1.9
98
Prob(Omnibus):        0.000    Jarque-Bera (JB):          3374.0
03
Skew:                 1.788    Prob(JB):                  0.
00
Kurtosis:             9.532    Cond. No.                  2.11e+
04
=====

```

```

=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.11e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```

In [42]: t_statistic = model.tvalues['LotArea']
p_value = model.pvalues['LotArea']
coeff = model.params['LotArea']
std_err = model.bse['LotArea']

print("Hypothesis Testing for LotArea:")
print(f"Coefficient: {coeff}")

```

```
print(f"Standard Error: {std_err}")
print(f"T-Statistic: {t_statistic}")
print(f"P-Value: {p_value}")
```

Hypothesis Testing for LotArea:
Coefficient: 2.099971951707674
Standard Error: 0.2010575176984752
T-Statistic: 10.444632838136346
P-Value: 1.1231391549186041e-24

- $H_0 : \beta_1 = 0$ the coefficient of LotArea β_1 is 0, the variable does not have statistical impact on SalePrice
- $H_1 : \beta_1 \neq 0$ the coefficient of LotArea β_1 is not 0, the variable has the statistical impact on SalePrice

$$t = \frac{\text{Coefficient}}{\text{Standard Error}} = \frac{2.10}{0.201} = 10.445$$

The p-value for the t-test is 1.12×10^{-24} . Since the p-value is much smaller than a standard significance level $\alpha = 0.05$, we reject the null hypothesis.

There is strong evidence to conclude that LotArea has a statistically significant relationship with SalePrice. However, the low R^2 (goodness-of-fit) of 0.070 suggests the model has limited explanatory capabilities, as only 7% of the variability in SalePrice is explained by LotArea. The adjusted R^2 of 0.069 accounts for the single predictor in the model and slightly reduces the explained variance, reinforcing the conclusion that LotArea alone does not sufficiently explain SalePrice.

The conceptual difference between R^2 and adjusted R^2 is that R^2 measures the proportion of the variance in the dependent variable explained by the independent variable(s), without accounting for the number of predictors in the model, while adjusted (R^2) adjusts for the number of predictors, penalizing the addition of variables that do not improve the model fit significantly.

The condition number 2.11×10^4 is high, indicating potential numerical issues or multicollinearity. While multicollinearity is not directly relevant in this single-variable model, the high condition number could reflect scaling differences or the wide range of LotArea values. This should be carefully addressed in models with multiple predictors.

Task 1.4

If you wish to argue that MSSubClass is insignificant and use the model with dummies, then you have to check the simultaneous insignificance of all the dummies that stem from the factor variable MSSubClass. Run a test for a

general linear hypothesis and conclude about the significance of `MSSubClass`. Write down the matrix and the vector needed in the hypothesis.

```
In [45]: # Names of the dummy variables, excluding the intercept
dummy_vars = [col for col in X.columns if col != 'const']

# Restriction matrix R with zeros and ones in the columns corresponding to t
R = np.zeros((len(dummy_vars), X.shape[1]))
for i, var in enumerate(dummy_vars):
    R[i, X.columns.get_loc(var)] = 1

# Null hypothesis vector q of zeros
q = np.zeros(len(dummy_vars))

# F-test to check the joint significance of all dummy variables
f_test = model.f_test((R, q))

print("F-Test for Joint Significance of MSSubClass Dummies:")
print(f"F-Statistic: {f_test.fvalue}")
print(f"P-Value: {f_test.pvalue}")
print(f"Degrees of Freedom (numerator): {f_test.df_num}")
print(f"Degrees of Freedom (denominator): {f_test.df_denom}")
```

F-Test for Joint Significance of MSSubClass Dummies:

F-Statistic: 109.09035512347613

P-Value: 1.123139154918582e-24

Degrees of Freedom (numerator): 1.0

Degrees of Freedom (denominator): 1458.0

- H_0 : all dummy variable coefficients for `MSSubClass` are zero
 $\beta_1 = \beta_2 = \dots = \beta_k = 0$; this implies that `MSSubClass` has no effect on `SalePrice`
- H_1 : at least one dummy variable coefficient is not zero $\exists \beta_i \neq 0$ and `MSSubClass` significantly affects `SalePrice`

F-Test Results:

1. F-statistic: 109.09
2. p-value: 1.12×10^{-24}
3. Degrees of freedom (nominator): 1.0
4. Degrees of freedom (denominator): 1458.0

The F-statistic is very large, and the p-value is extremely small. This provides strong evidence to reject the null hypothesis, indicating that `MSSubClass` is significant in explaining variability in `SalePrice`.

1. Restriction Matrix (R):

- A $k \times (p + 1)$ matrix, where k is the number of dummy variables for `MSSubClass` and $p + 1$ is the total number of coefficients in the model,

including the intercept.

- Each row of R corresponds to one dummy variable's coefficient and contains a 1 in the column corresponding to that coefficient and 0 elsewhere.
- For example for two dummies:

$$R = \begin{bmatrix} 0 & 1 & 0 & \dots & 0 \\ 0 & 0 & 1 & \dots & 0 \end{bmatrix}$$

2. Null Hypothesis Vector (q) is a $k \times 1$ vector of zeros:

$$q = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

The dummy variables for `MSSubClass` collectively explain a meaningful portion of the variation in `SalePrice`, and therefore, `MSSubClass` should not be excluded from the model. Testing all dummies together ensures we capture the overall significance of the factor variable, even if individual dummies may seem insignificant.

Task 1.5

Give economic interpretation for the parameters of `LotArea`, the first dummy for `MSSubClass` (it is a tricky question!), and `BedroomAbvGr`. (Ignore the potential insignificance.) How would the interpretations change if you model the log of `SalePrice`?

```
In [5]: # Build simple linear regression models for key variables to see how they ma

# Linear model for SalePrice
X = data[['LotArea', 'BedroomAbvGr']]
X = sm.add_constant(X)
y = data['SalePrice']
model_linear = sm.OLS(y, X).fit()
print(model_linear.summary())

# Log-linear model for SalePrice
y_log = np.log(data['SalePrice'])
model_log_linear = sm.OLS(y_log, X).fit()
print(model_log_linear.summary())
```

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice  R-squared:                0.0
89
Model:                  OLS      Adj. R-squared:              0.0
87
Method:                 Least Squares  F-statistic:              70.
78
Date:                   Sun, 22 Dec 2024  Prob (F-statistic):      4.62e-
30
Time:                   21:17:06      Log-Likelihood:          -1847
6.
No. Observations:      1460      AIC:                      3.696e+
04
Df Residuals:          1457      BIC:                      3.697e+
04
Df Model:               2
Covariance Type:       nonrobust
=====

```

```

=====
=====
              coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const          1.215e+05    7368.475     16.493     0.000     1.07e+05     1.36
e+05
LotArea         1.9679         0.201      9.815     0.000         1.575
2.361
BedroomAbvGr    1.35e+04    2453.296      5.502     0.000     8686.751     1.83
e+04
=====

```

```

==
Omnibus:              597.485  Durbin-Watson:              2.0
06
Prob(Omnibus):        0.000  Jarque-Bera (JB):      3369.7
77
Skew:                 1.831  Prob(JB):              0.
00
Kurtosis:             9.479  Cond. No.              5.63e+
04
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.63e+04. This might indicate that there are strong multicollinearity or other numerical problems.

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice  R-squared:                0.0
98
Model:                  OLS      Adj. R-squared:              0.0

```



```

97
Method:                Least Squares    F-statistic:                79.
55
Date:                  Sun, 22 Dec 2024  Prob (F-statistic):        1.63e-
33
Time:                  21:17:06          Log-Likelihood:            -655.
71
No. Observations:      1460             AIC:                        131
7.
Df Residuals:          1457             BIC:                        133
3.
Df Model:              2
Covariance Type:       nonrobust

```

```

=====
=====
              coef      std err          t      P>|t|      [0.025      0.
975]
-----
----
const          11.6710      0.037     316.731     0.000      11.599      1
1.743
LotArea        9.432e-06      1e-06      9.406     0.000      7.46e-06      1.14
e-05
BedroomAbvGr   0.0885      0.012      7.217     0.000      0.064
0.113
=====
==
Omnibus:                25.579    Durbin-Watson:                1.9
74
Prob(Omnibus):          0.000    Jarque-Bera (JB):                40.5
20
Skew:                   0.148    Prob(JB):                        1.59e-
09
Kurtosis:               3.761    Cond. No.                        5.63e+
04
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 5.63e+04. This might indicate that there are strong multicollinearity or other numerical problems.

The economic interpretation of `LotArea`, the first dummy for `MSSubClass`, and `BedroomAbvGr` varies depending on whether we use a linear or log-linear model. In the **linear model**, the parameter for `LotArea` (e.g., 1.967) indicates that an additional square foot of lot size increases the sale price by 1.97, holding other factors constant. Similarly, the parameter for `BedroomAbvGr` (e.g., 13,500) means that adding one more bedroom increases the sale price by 13,500. For the first dummy of `MSSubClass`, the coefficient represents the dollar difference in sale price compared to the reference category. For example,

if the coefficient is -10,000, it implies that properties in this category sell for \$10,000 less than those in the reference category.

In the **log-linear model**, the interpretation shifts to proportional changes. The parameter for `LotArea` (e.g., 9.43e-6) implies a 0.000943% increase in sale price for each additional square foot of lot size. The coefficient for `BedroomAbvGr` (e.g., 0.0885) means that adding one bedroom increases the sale price by approximately 8.85%. Similarly, the first dummy for `MSSubClass` (e.g., -0.05) indicates that properties in this category sell for about 5% less than those in the reference category.

The key difference lies in the type of change measured: the linear model captures **absolute changes in dollars**, while the log-linear model captures **relative changes in percentages**, which are more suitable for comparing properties with varying price scales.

Task 1.6

Compute the 95% confidence interval for the parameter of `LotArea` and provide its economic meaning. Relate the CIs to the tests of significance, i.e., how would you use these intervals to decide about the significance of the corresponding explanatory variables? The CIs are computed relying on the assumption that the residuals follow a normal distribution. Is this assumption fulfilled? Run an appropriate goodness-of-fit test.

The results from the previous linear regression model can be used to compute the 95% confidence interval for the `LotArea` coefficient:

$$CI = \beta \pm t_{\alpha/2} \cdot SE(\beta)$$

Where:

- $\beta = 1.967$: coefficient estimate for `LotArea`
- $t_{\alpha/2}$: critical value for the t-distribution at a 95% confidence level
- $SE(\beta) = 0.201$: standard error of the coefficient
- $df = 1457$: degrees of freedom

```
In [7]: beta = 1.967
se_beta = 0.201
t_critical = stats.t.ppf(1 - 0.025, df=1457)

lower_bound = beta - t_critical * se_beta
upper_bound = beta + t_critical * se_beta

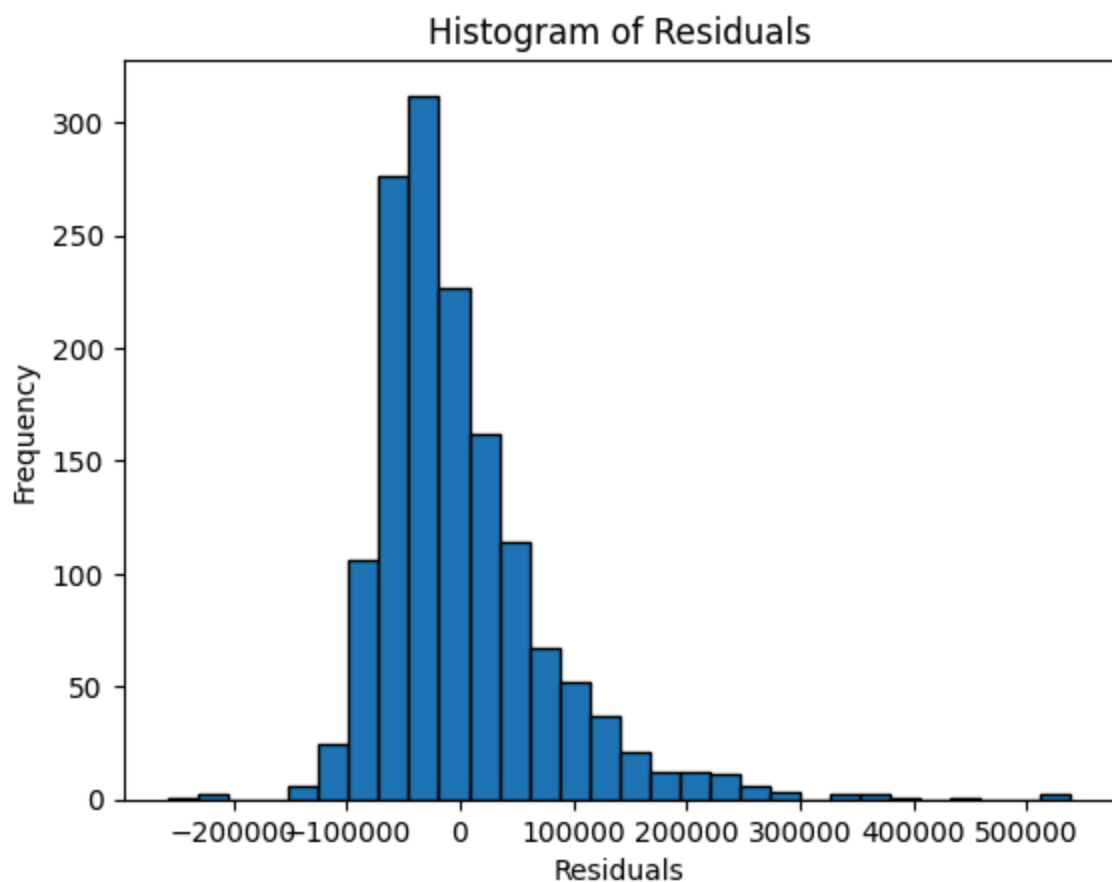
print(f"95% Confidence Interval for LotArea: [{lower_bound:.3f}, {upper_bour
```

95% Confidence Interval for LotArea: [1.573, 2.361]

The confidence interval indicates that for every additional square foot of lot size, the sale price is expected to increase by 1.575 to 2.361 with 95% confidence.

But the validity of the confidence interval relies on the assumption that residuals are normally distributed. Let's check this.

```
In [9]: # Residuals histogram
residuals = model_linear.resid
plt.hist(residuals, bins=30, edgecolor='k')
plt.title("Histogram of Residuals")
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.show()
```



```
In [10]: # Shapiro-Wilk test
stat, p_value = stats.shapiro(residuals)
print(f"Shapiro-Wilk Test Statistic: {stat:.3f}, P-Value: {p_value:.3f}")
```

Shapiro-Wilk Test Statistic: 0.871, P-Value: 0.000

The histogram shows that the residuals are approximately centered around zero, but the distribution appears to have noticeable skewness to the right. This suggests that the residuals deviate from a perfectly normal distribution.

Additionally, since the p-value is less than 0.05 in Shapiro-Wilk test, we reject the null hypothesis that the residuals follow a normal distribution.

Let's try modeling the log of `SalePrice` to improve normality and stabilize variance.

```
In [12]: # Log transformation to SalePrice
data['LogSalePrice'] = np.log(data['SalePrice'])

X_simple = data[['LotArea']]
X_simple = sm.add_constant(X_simple)
y_log = data['LogSalePrice']
model_log_simple = sm.OLS(y_log, X_simple).fit()

print(model_log_simple.summary())
```

OLS Regression Results

```

=====
==
Dep. Variable:          LogSalePrice    R-squared:                0.0
66
Model:                  OLS    Adj. R-squared:              0.0
66
Method:                Least Squares    F-statistic:              10
3.4
Date:                  Sun, 22 Dec 2024    Prob (F-statistic):      1.64e-
23
Time:                  21:48:25    Log-Likelihood:          -681.
35
No. Observations:      1460    AIC:                      136
7.
Df Residuals:          1458    BIC:                      137
7.
Df Model:              1
Covariance Type:      nonrobust
=====

```

```

=====
==
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const         11.9157      0.015     811.561      0.000      11.887      11.9
45
LotArea       1.03e-05     1.01e-06     10.168      0.000      8.31e-06     1.23e-
05
=====

```

```

=====
==
Omnibus:          25.294    Durbin-Watson:           1.9
60
Prob(Omnibus):    0.000    Jarque-Bera (JB):        47.3
35
Skew:             0.036    Prob(JB):                5.26e-
11
Kurtosis:         3.879    Cond. No.                 2.11e+
04
=====

```

```

=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.11e+04. This might indicate that there are strong multicollinearity or other numerical problems.

```

In [21]: # Log-transformed SalePrice and rescaled LotArea
data['LotArea_scaled'] = data['LotArea'] / 1000
data['LogSalePrice'] = np.log(data['SalePrice'])

X_scaled = data[['LotArea_scaled']]
X_scaled = sm.add_constant(X_scaled)
y_log = np.log(data['LogSalePrice'])

```

```
model_log_scaled = sm.OLS(y_log, X_scaled).fit()
print(model_log_simple.summary())
```

OLS Regression Results

```
=====
==
Dep. Variable:          LogSalePrice    R-squared:                0.0
66
Model:                  OLS            Adj. R-squared:           0.0
66
Method:                 Least Squares   F-statistic:              10
3.4
Date:                   Sun, 22 Dec 2024 Prob (F-statistic):       1.64e-
23
Time:                   22:02:36        Log-Likelihood:          -681.
35
No. Observations:       1460           AIC:                      136
7.
Df Residuals:           1458           BIC:                      137
7.
Df Model:                1
Covariance Type:        nonrobust
=====
==
               coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const          11.9157      0.015     811.561      0.000      11.887      11.9
45
LotArea        1.03e-05    1.01e-06     10.168      0.000      8.31e-06    1.23e-
05
=====
==
Omnibus:                25.294    Durbin-Watson:           1.9
60
Prob(Omnibus):           0.000    Jarque-Bera (JB):         47.3
35
Skew:                   0.036    Prob(JB):                 5.26e-
11
Kurtosis:                3.879    Cond. No.                 2.11e+
04
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 2.11e+04. This might indicate that there are strong multicollinearity or other numerical problems.

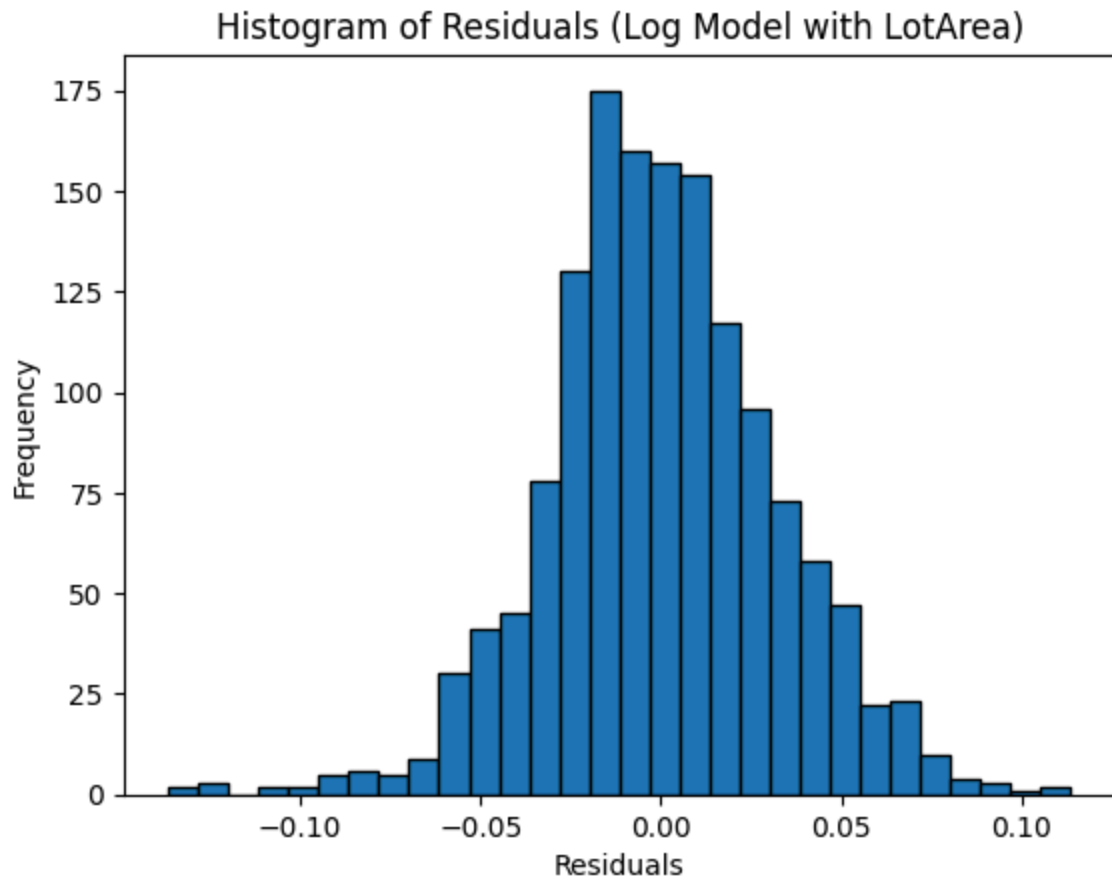
```
In [22]: # Residuals check
residuals_log_simple = model_log_scaled.resid

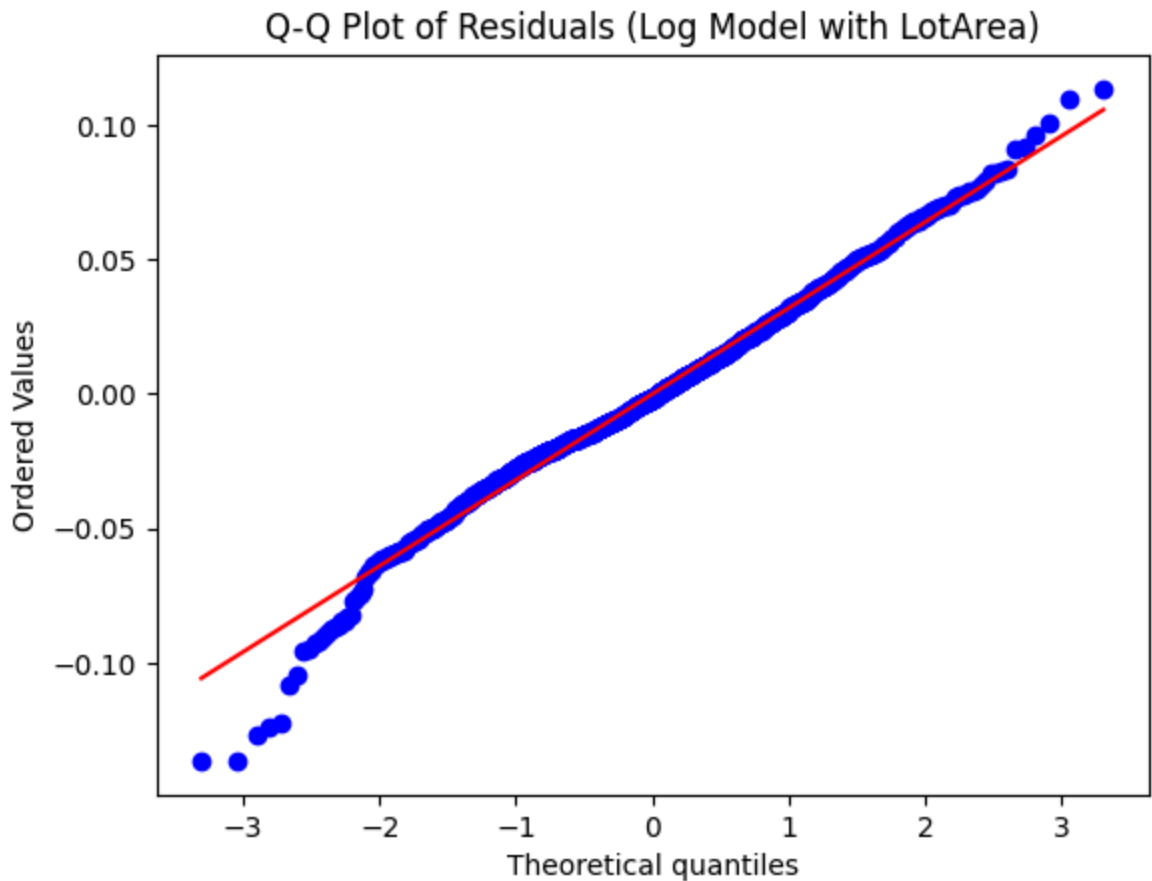
# Residuals histogram
```

```
plt.hist(residuals_log_simple, bins=30, edgecolor='k')
plt.title("Histogram of Residuals (Log Model with LotArea)")
plt.xlabel("Residuals")
plt.ylabel("Frequency")
plt.show()

# Q-Q plot
stats.probplot(residuals_log_simple, dist="norm", plot=plt)
plt.title("Q-Q Plot of Residuals (Log Model with LotArea)")
plt.show()

# Shapiro-Wilk test
stat, p_value = stats.shapiro(residuals_log_simple)
print(f"Shapiro-Wilk Test Statistic: {stat:.3f}, P-Value: {p_value:.3f}")
```





Shapiro-Wilk Test Statistic: 0.990, P-Value: 0.000

Although the Shapiro-Wilk test rejects the hypothesis of normality due to slight deviations in the tails and the test's sensitivity to large sample sizes, the Q-Q plot and histogram show a nearly symmetric, bell-shaped distribution, indicating that the residuals are close enough to normal for regression assumptions.

```
In [23]: # Confidence interval
beta_log_scaled = model_log_scaled.params['LotArea_scaled']
se_beta_log_scaled = model_log_scaled.bse['LotArea_scaled']

t_critical_log_scaled = stats.t.ppf(1 - 0.025, df=model_log_scaled.df_resid)

lower_bound_log_scaled = beta_log_scaled - t_critical_log_scaled * se_beta_log_scaled
upper_bound_log_scaled = beta_log_scaled + t_critical_log_scaled * se_beta_log_scaled

print(f"95% Confidence Interval for LotArea (Log Model with Scaling): "
      f"[{lower_bound_log_scaled:.5f}, {upper_bound_log_scaled:.5f}]")
```

95% Confidence Interval for LotArea (Log Model with Scaling): [0.00068, 0.00101]

From this confidence interval can be seen that for every 1-unit increase in the scaled `LotArea` (equivalent to 1,000 square feet), the natural logarithm of the `SalePrice` is expected to increase by a value between 0.00068 and 0.00101. Economically, it represents the percentage change in `SalePrice` for every 1,000 square feet increase in `LotArea`.

Task 1.7

Most of the variables appear to be insignificant, and we should find the smallest model that still has good explanatory power. Choose this model using stepwise model selection (either based on the tests for R^2 or using adjusted R^2 / AIC/ BIC). Pick the last step of the model selection procedure and explain in detail how the method/approach works (or is implemented in your software). Work with this model in all the remaining steps.

We will start with a full model that includes all available predictors and remove variables iteratively based on AIC values.

```
In [37]: # The full model with all predictors
X_full = data.drop(columns=["SalePrice"])

# Replace infinite values with NaN
X_full = X_full.replace([np.inf, -np.inf], np.nan)

# Handle missing values
for column in X_full.columns:
    if X_full[column].dtype in ['float64', 'int64']:
        X_full[column] = X_full[column].fillna(X_full[column].mean())
    elif X_full[column].dtype == 'bool':
        X_full[column] = X_full[column].astype(int)
    else:
        X_full[column] = X_full[column].fillna("Missing")

# Encode categorical variables
X_full_encoded = pd.get_dummies(X_full, drop_first=True)

# Fix data types
X_full_encoded = X_full_encoded.astype('float64')
X_full_encoded = sm.add_constant(X_full_encoded)

# Debugging and fitting stage
print("Final dtypes of predictors after processing:", X_full_encoded.dtypes)
y = pd.to_numeric(data["SalePrice"], errors="coerce").fillna(0)
print("Final dtype of target variable:", y.dtype)

try:
    full_model = sm.OLS(y, X_full_encoded).fit()
    print(full_model.summary())
except ValueError as e:
    print("Error in fitting the model:", e)
```

Final dtypes of predictors after processing: [dtype('float64')]

Final dtype of target variable: int64

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice  R-squared:                0.9
69
Model:                  OLS      Adj. R-squared:              0.9
62
Method:                 Least Squares  F-statistic:           14
7.6
Date:                   Sun, 22 Dec 2024  Prob (F-statistic):      0.
00
Time:                   22:24:53   Log-Likelihood:         -1601
1.
No. Observations:      1460      AIC:                    3.253e+
04
Df Residuals:          1205      BIC:                    3.388e+
04
Df Model:               254
Covariance Type:       nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.02
5      0.975]
-----
const                -1.67e+06    7.23e+05    -2.309    0.021    -3.09e+0
6    -2.51e+05
Id                   2.0056        1.057        1.898    0.058    -0.06
8           4.079
MSSubClass           10.8909        56.505        0.193    0.847    -99.96
8           121.750
LotFrontage          -16.1730        30.087       -0.538    0.591    -75.20
1           42.855
LotArea               0.2557         0.076        3.375    0.001         0.10
7           0.404
OverallQual           326.2166       714.489        0.457    0.648   -1075.56
4           1727.997
OverallCond           15.9400        615.159        0.026    0.979   -1190.96
2           1222.842
YearBuilt             48.0808        53.142        0.905    0.366    -56.18
1           152.342
YearRemodAdd          -29.4077        38.285       -0.768    0.443   -104.52
1           45.705
MasVnrArea            19.6549         3.947        4.980    0.000        11.91
2           27.398
BsmtFinSF1            7.3711         1.988        3.708    0.000         3.47
1           11.271
BsmtFinSF2            1.6163         4.082        0.396    0.692    -6.39
3           9.625
BsmtUnfSF             -0.9896         1.892       -0.523    0.601    -4.70
1           2.722
TotalBsmtSF           7.9978         2.804        2.852    0.004         2.49
6           13.499
1stFlrSF              7.7431         4.371        1.771    0.077    -0.83
```

2	16.319					
2ndFlrSF		26.5038	3.742	7.084	0.000	19.16
3	33.844					
LowQualFinSF		-34.2528	9.508	-3.603	0.000	-52.90
6	-15.600					
GrLivArea		-0.0059	3.955	-0.001	0.999	-7.76
5	7.753					
BsmtFullBath		-2400.1094	1355.636	-1.770	0.077	-5059.77
8	259.559					
BsmtHalfBath		-1222.1834	2066.548	-0.591	0.554	-5276.61
6	2832.250					
FullBath		708.9065	1503.722	0.471	0.637	-2241.29
9	3659.111					
HalfBath		-1785.7338	1434.462	-1.245	0.213	-4600.05
4	1028.586					
BedroomAbvGr		-4708.1269	932.075	-5.051	0.000	-6536.79
8	-2879.456					
KitchenAbvGr		-6621.1580	3887.585	-1.703	0.089	-1.42e+0
4	1006.030					
TotRmsAbvGrd		1081.4289	652.937	1.656	0.098	-199.59
1	2362.448					
Fireplaces		4391.6724	1745.201	2.516	0.012	967.70
3	7815.642					
GarageYrBlt		25.1323	41.862	0.600	0.548	-56.99
9	107.263					
GarageCars		1288.7264	1556.913	0.828	0.408	-1765.83
6	4343.289					
GarageArea		-2.1458	5.421	-0.396	0.692	-12.78
1	8.490					
WoodDeckSF		0.0975	4.033	0.024	0.981	-7.81
4	8.009					
OpenPorchSF		-4.5872	7.901	-0.581	0.562	-20.08
9	10.914					
EnclosedPorch		-16.7701	8.534	-1.965	0.050	-33.51
3	-0.027					
3SsnPorch		10.3087	15.296	0.674	0.500	-19.70
2	40.319					
ScreenPorch		-8.0940	8.614	-0.940	0.348	-24.99
4	8.806					
PoolArea		416.5390	154.996	2.687	0.007	112.44
7	720.631					
MiscVal		0.9629	4.176	0.231	0.818	-7.23
1	9.157					
MoSold		-388.6853	167.291	-2.323	0.020	-716.89
9	-60.471					
YrSold		-196.9689	351.787	-0.560	0.576	-887.15
2	493.214					
LogSalePrice		1.587e+05	4274.183	37.120	0.000	1.5e+0
5	1.67e+05					
LotArea_scaled		0.0003	7.58e-05	3.375	0.001	0.00
0	0.000					
MSZoning_FV		-3.842e+04	8414.058	-4.566	0.000	-5.49e+0
4	-2.19e+04					
MSZoning_RH		-4.589e+04	8325.908	-5.512	0.000	-6.22e+0
4	-2.96e+04					
MSZoning_RL		-4.212e+04	7210.572	-5.841	0.000	-5.63e+0

4	-2.8e+04					
MSZoning_RM	-3.93e+04	6748.343	-5.824	0.000	-5.25e+0	
4	-2.61e+04					
Street_Pave	1.774e+04	8332.788	2.129	0.033	1394.30	
1	3.41e+04					
Alley_Missing	309.8484	2877.465	0.108	0.914	-5335.55	
1	5955.248					
Alley_Pave	-2686.1639	4115.001	-0.653	0.514	-1.08e+0	
4	5387.199					
LotShape_IR2	988.7737	2877.970	0.344	0.731	-4657.61	
6	6635.163					
LotShape_IR3	2604.5660	6056.418	0.430	0.667	-9277.72	
9	1.45e+04					
LotShape_Reg	588.0970	1094.604	0.537	0.591	-1559.44	
4	2735.638					
LandContour_HLS	2969.4078	3498.043	0.849	0.396	-3893.52	
4	9832.340					
LandContour_Low	-8549.6739	4361.340	-1.960	0.050	-1.71e+0	
4	6.991					
LandContour_Lvl	1014.2537	2530.149	0.401	0.689	-3949.73	
4	5978.241					
Utilities_NoSeWa	-13.6442	1.8e+04	-0.001	0.999	-3.54e+0	
4	3.54e+04					
LotConfig_CulDSac	4089.1709	2257.676	1.811	0.070	-340.24	
2	8518.584					
LotConfig_FR2	-1765.6836	2748.497	-0.642	0.521	-7158.05	
5	3626.687					
LotConfig_FR3	-2499.9830	8584.882	-0.291	0.771	-1.93e+0	
4	1.43e+04					
LotConfig_Inside	1042.0762	1223.674	0.852	0.395	-1358.69	
1	3442.844					
LandSlope_Mod	2434.5180	2718.289	0.896	0.371	-2898.58	
7	7767.623					
LandSlope_Sev	-1.029e+04	7839.826	-1.312	0.190	-2.57e+0	
4	5091.541					
Neighborhood_Blueste	1.41e+04	1.31e+04	1.074	0.283	-1.17e+0	
4	3.99e+04					
Neighborhood_BrDale	9091.2734	7490.364	1.214	0.225	-5604.33	
1	2.38e+04					
Neighborhood_BrkSide	-5104.2922	6469.204	-0.789	0.430	-1.78e+0	
4	7587.863					
Neighborhood_ClearCr	-1.729e+04	6284.534	-2.751	0.006	-2.96e+0	
4	-4957.772					
Neighborhood_CollgCr	-6189.1533	4950.268	-1.250	0.211	-1.59e+0	
4	3522.950					
Neighborhood_Crawfor	-5319.0644	5849.526	-0.909	0.363	-1.68e+0	
4	6157.323					
Neighborhood_Edwards	-6485.6579	5468.474	-1.186	0.236	-1.72e+0	
4	4243.130					
Neighborhood_Gilbert	-8432.1086	5238.758	-1.610	0.108	-1.87e+0	
4	1845.991					
Neighborhood_IDOTRR	-5606.3143	7332.789	-0.765	0.445	-2e+0	
4	8780.138					
Neighborhood_MeadowV	2.123e+04	7675.040	2.766	0.006	6170.44	
7	3.63e+04					
Neighborhood_Mitchel	-1.133e+04	5580.494	-2.030	0.043	-2.23e+0	

4	-377.915					
Neighborhood_NAmes	-9900.2638	5348.319	-1.851	0.064	-2.04e+0	
4	592.788					
Neighborhood_NPkVill	1.33e+04	9585.420	1.387	0.166	-5508.96	
7	3.21e+04					
Neighborhood_NWAmes	-9898.7036	5467.988	-1.810	0.070	-2.06e+0	
4	829.131					
Neighborhood_NoRidge	2.128e+04	5744.657	3.705	0.000	1e+0	
4	3.26e+04					
Neighborhood_NridgHt	6894.6528	5133.615	1.343	0.180	-3177.16	
4	1.7e+04					
Neighborhood_OldTown	-5099.4933	6593.353	-0.773	0.439	-1.8e+0	
4	7836.234					
Neighborhood_SWISU	-7683.6503	6621.381	-1.160	0.246	-2.07e+0	
4	5307.066					
Neighborhood_Sawyer	-5276.1923	5540.909	-0.952	0.341	-1.61e+0	
4	5594.709					
Neighborhood_SawyerW	-1117.9261	5307.899	-0.211	0.833	-1.15e+0	
4	9295.825					
Neighborhood_Somerst	-4836.5028	6146.780	-0.787	0.432	-1.69e+0	
4	7223.077					
Neighborhood_StoneBr	1.879e+04	5679.885	3.309	0.001	7648.60	
8	2.99e+04					
Neighborhood_Timber	-9482.6854	5553.079	-1.708	0.088	-2.04e+0	
4	1412.092					
Neighborhood_Veenker	-6445.6494	7159.936	-0.900	0.368	-2.05e+0	
4	7601.678					
Condition1_Feedr	546.8409	3428.941	0.159	0.873	-6180.51	
7	7274.199					
Condition1_Norm	2014.6099	2883.700	0.699	0.485	-3643.02	
1	7672.241					
Condition1_PosA	3982.5810	6831.263	0.583	0.560	-9419.91	
0	1.74e+04					
Condition1_PosN	171.2045	5089.863	0.034	0.973	-9814.77	
3	1.02e+04					
Condition1_RRAe	-9849.9658	6189.173	-1.591	0.112	-2.2e+0	
4	2292.787					
Condition1_RRAn	3289.2337	4752.501	0.692	0.489	-6034.86	
2	1.26e+04					
Condition1_RRNe	-5832.3072	1.19e+04	-0.489	0.625	-2.92e+0	
4	1.76e+04					
Condition1_RRNn	-2939.6874	8770.563	-0.335	0.738	-2.01e+0	
4	1.43e+04					
Condition2_Feedr	-2.308e+04	1.6e+04	-1.443	0.149	-5.44e+0	
4	8294.319					
Condition2_Norm	-1.708e+04	1.38e+04	-1.233	0.218	-4.42e+0	
4	1.01e+04					
Condition2_PosA	4356.6352	2.53e+04	0.172	0.863	-4.53e+0	
4	5.4e+04					
Condition2_PosN	-1.065e+05	1.92e+04	-5.547	0.000	-1.44e+0	
5	-6.88e+04					
Condition2_RRAe	-3.073e+04	4.45e+04	-0.691	0.490	-1.18e+0	
5	5.66e+04					
Condition2_RRAn	-1.432e+04	2.15e+04	-0.667	0.505	-5.65e+0	
4	2.78e+04					
Condition2_RRNn	-6820.7067	1.85e+04	-0.369	0.712	-4.31e+0	

4	2.95e+04				
BldgType_2fmCon	-1.127e+04	8528.462	-1.322	0.186	-2.8e+0
4	5458.830				
BldgType_Duplex	-7494.2114	5069.130	-1.478	0.140	-1.74e+0
4	2451.090				
BldgType_Twnhs	-1.182e+04	6843.097	-1.728	0.084	-2.53e+0
4	1601.380				
BldgType_TwnhsE	-1.358e+04	6168.650	-2.202	0.028	-2.57e+0
4	-1478.118				
HouseStyle_1.5Unf	1.115e+04	5436.749	2.052	0.040	488.27
9	2.18e+04				
HouseStyle_1Story	9994.0868	2998.918	3.333	0.001	4110.40
6	1.59e+04				
HouseStyle_2.5Fin	-1470.5732	8462.717	-0.174	0.862	-1.81e+0
4	1.51e+04				
HouseStyle_2.5Unf	-1.439e+04	6306.887	-2.282	0.023	-2.68e+0
4	-2016.582				
HouseStyle_2Story	-2254.0283	2390.868	-0.943	0.346	-6944.75
5	2436.699				
HouseStyle_SFoyer	2679.1461	4275.622	0.627	0.531	-5709.34
6	1.11e+04				
HouseStyle_SLvl	4115.6110	3812.965	1.079	0.281	-3365.17
7	1.16e+04				
RoofStyle_Gable	8150.1773	1.26e+04	0.647	0.517	-1.65e+0
4	3.28e+04				
RoofStyle_Gambrel	1.13e+04	1.38e+04	0.820	0.412	-1.57e+0
4	3.83e+04				
RoofStyle_Hip	7509.4570	1.26e+04	0.594	0.552	-1.73e+0
4	3.23e+04				
RoofStyle_Mansard	1.05e+04	1.46e+04	0.718	0.473	-1.82e+0
4	3.92e+04				
RoofStyle_Shed	2.662e+04	2.37e+04	1.126	0.261	-1.98e+0
4	7.3e+04				
RoofMatl_CompShg	1.674e+05	3.77e+04	4.444	0.000	9.35e+0
4	2.41e+05				
RoofMatl_Membran	1.982e+05	4.46e+04	4.441	0.000	1.11e+0
5	2.86e+05				
RoofMatl_Metal	1.873e+05	4.42e+04	4.235	0.000	1.01e+0
5	2.74e+05				
RoofMatl_Roll	1.504e+05	4.14e+04	3.637	0.000	6.93e+0
4	2.32e+05				
RoofMatl_Tar&Grv	1.674e+05	4.02e+04	4.169	0.000	8.86e+0
4	2.46e+05				
RoofMatl_WdShake	1.683e+05	3.91e+04	4.300	0.000	9.15e+0
4	2.45e+05				
RoofMatl_WdShngl	2.137e+05	3.84e+04	5.569	0.000	1.38e+0
5	2.89e+05				
Exterior1st_AsphShn	-2.254e+04	2.25e+04	-1.000	0.317	-6.67e+0
4	2.17e+04				
Exterior1st_BrkComm	3.066e+04	1.9e+04	1.615	0.107	-6592.46
5	6.79e+04				
Exterior1st_BrkFace	-3673.1624	8725.706	-0.421	0.674	-2.08e+0
4	1.34e+04				
Exterior1st_CBlock	-3306.1623	9311.338	-0.355	0.723	-2.16e+0
4	1.5e+04				
Exterior1st_CemntBd	4984.4144	1.3e+04	0.383	0.702	-2.05e+0

4	3.05e+04					
Exterior1st_HdBoard	-8556.8300	8840.743	-0.968	0.333	-2.59e+0	
4	8788.130					
Exterior1st_ImStucc	-2.453e+04	1.92e+04	-1.276	0.202	-6.23e+0	
4	1.32e+04					
Exterior1st_MetalSd	-1.163e+04	9973.291	-1.166	0.244	-3.12e+0	
4	7941.729					
Exterior1st_Plywood	-1.048e+04	8724.612	-1.201	0.230	-2.76e+0	
4	6638.141					
Exterior1st_Stone	-2441.4354	1.66e+04	-0.147	0.883	-3.5e+0	
4	3.01e+04					
Exterior1st_Stucco	-8111.7645	9618.715	-0.843	0.399	-2.7e+0	
4	1.08e+04					
Exterior1st_VinylSd	-1.099e+04	9112.445	-1.206	0.228	-2.89e+0	
4	6889.612					
Exterior1st_Wd Sdng	-5009.8444	8461.160	-0.592	0.554	-2.16e+0	
4	1.16e+04					
Exterior1st_WdShing	-6796.4239	9130.687	-0.744	0.457	-2.47e+0	
4	1.11e+04					
Exterior2nd_AsphShn	896.6850	1.52e+04	0.059	0.953	-2.89e+0	
4	3.07e+04					
Exterior2nd_Brk Cmn	-3780.5546	1.37e+04	-0.276	0.783	-3.07e+0	
4	2.31e+04					
Exterior2nd_BrkFace	5747.8018	9030.383	0.636	0.525	-1.2e+0	
4	2.35e+04					
Exterior2nd_CBlock	-3306.1623	9311.338	-0.355	0.723	-2.16e+0	
4	1.5e+04					
Exterior2nd_CmentBd	-1.329e+04	1.28e+04	-1.038	0.299	-3.84e+0	
4	1.18e+04					
Exterior2nd_HdBoard	1227.1475	8487.791	0.145	0.885	-1.54e+0	
4	1.79e+04					
Exterior2nd_ImStucc	9031.0303	9802.994	0.921	0.357	-1.02e+0	
4	2.83e+04					
Exterior2nd_MetalSd	4340.6269	9703.730	0.447	0.655	-1.47e+0	
4	2.34e+04					
Exterior2nd_0ther	-8541.2568	1.85e+04	-0.462	0.644	-4.48e+0	
4	2.77e+04					
Exterior2nd_Plywood	-143.3294	8241.469	-0.017	0.986	-1.63e+0	
4	1.6e+04					
Exterior2nd_Stone	-9011.6141	1.17e+04	-0.770	0.442	-3.2e+0	
4	1.4e+04					
Exterior2nd_Stucco	-687.6515	9302.280	-0.074	0.941	-1.89e+0	
4	1.76e+04					
Exterior2nd_VinylSd	3571.3752	8760.099	0.408	0.684	-1.36e+0	
4	2.08e+04					
Exterior2nd_Wd Sdng	-725.7419	8169.777	-0.089	0.929	-1.68e+0	
4	1.53e+04					
Exterior2nd_Wd Shng	-1230.5085	8515.144	-0.145	0.885	-1.79e+0	
4	1.55e+04					
MasVnrType_BrkFace	-1919.4123	4669.406	-0.411	0.681	-1.11e+0	
4	7241.657					
MasVnrType_Missing	2383.9039	4715.595	0.506	0.613	-6867.78	
6	1.16e+04					
MasVnrType_Stone	1530.1948	4947.169	0.309	0.757	-8175.82	
7	1.12e+04					
ExterQual_Fa	-9721.7395	7570.903	-1.284	0.199	-2.46e+0	

4	5131.878					
ExterQual_Gd	-2.065e+04	3263.762	-6.328	0.000	-2.71e+0	
4	-1.42e+04					
ExterQual_TA	-2.103e+04	3619.451	-5.810	0.000	-2.81e+0	
4	-1.39e+04					
ExterCond_Fa	1.41e+04	1.23e+04	1.142	0.254	-1.01e+0	
4	3.83e+04					
ExterCond_Gd	5823.6106	1.18e+04	0.495	0.621	-1.73e+0	
4	2.89e+04					
ExterCond_Po	1.616e+04	2.16e+04	0.747	0.455	-2.63e+0	
4	5.86e+04					
ExterCond_TA	5234.5536	1.17e+04	0.446	0.656	-1.78e+0	
4	2.83e+04					
Foundation_CBlock	-332.1238	2167.325	-0.153	0.878	-4584.27	
5	3920.027					
Foundation_PConc	-2049.7645	2339.305	-0.876	0.381	-6639.32	
9	2539.800					
Foundation_Slab	-3820.4822	6859.722	-0.557	0.578	-1.73e+0	
4	9637.845					
Foundation_Stone	-7220.7254	7798.411	-0.926	0.355	-2.25e+0	
4	8079.248					
Foundation_Wood	-8217.1758	1.01e+04	-0.814	0.416	-2.8e+0	
4	1.16e+04					
BsmtQual_Fa	-7726.5488	4340.501	-1.780	0.075	-1.62e+0	
4	789.231					
BsmtQual_Gd	-1.371e+04	2280.655	-6.012	0.000	-1.82e+0	
4	-9237.454					
BsmtQual_Missing	3533.9161	8337.804	0.424	0.672	-1.28e+0	
4	1.99e+04					
BsmtQual_TA	-9710.4184	2841.433	-3.417	0.001	-1.53e+0	
4	-4135.713					
BsmtCond_Gd	-4438.9268	3603.637	-1.232	0.218	-1.15e+0	
4	2631.172					
BsmtCond_Missing	3533.9161	8337.804	0.424	0.672	-1.28e+0	
4	1.99e+04					
BsmtCond_Po	2.006e+04	2.04e+04	0.983	0.326	-2e+0	
4	6.01e+04					
BsmtCond_TA	-1076.9329	2899.945	-0.371	0.710	-6766.43	
5	4612.569					
BsmtExposure_Gd	9140.4302	2050.855	4.457	0.000	5116.78	
8	1.32e+04					
BsmtExposure_Missing	-3074.8816	1.57e+04	-0.196	0.845	-3.39e+0	
4	2.77e+04					
BsmtExposure_Mn	-2520.7988	2060.835	-1.223	0.221	-6564.02	
3	1522.425					
BsmtExposure_No	-3428.8736	1488.718	-2.303	0.021	-6349.64	
2	-508.106					
BsmtFinType1_BLQ	2922.3049	1913.984	1.527	0.127	-832.80	
6	6677.416					
BsmtFinType1_GLQ	3385.1318	1721.596	1.966	0.049	7.47	
2	6762.791					
BsmtFinType1_LwQ	720.7138	2557.745	0.282	0.778	-4297.41	
5	5738.842					
BsmtFinType1_Missing	3533.9161	8337.804	0.424	0.672	-1.28e+0	
4	1.99e+04					
BsmtFinType1_Rec	1430.7609	2048.871	0.698	0.485	-2588.99	

0	5450.512					
BsmtFinType1_Unf	4452.6611	1991.328	2.236	0.026	545.80	
5	8359.517					
BsmtFinType2_BLQ	-1817.3345	5173.774	-0.351	0.725	-1.2e+0	
4	8333.271					
BsmtFinType2_GLQ	-3308.4641	6381.962	-0.518	0.604	-1.58e+0	
4	9212.528					
BsmtFinType2_LwQ	-8739.3983	5051.966	-1.730	0.084	-1.87e+0	
4	1172.228					
BsmtFinType2_Missing	-5144.4650	1.71e+04	-0.302	0.763	-3.86e+0	
4	2.83e+04					
BsmtFinType2_Rec	-4976.0064	4861.880	-1.023	0.306	-1.45e+0	
4	4562.684					
BsmtFinType2_Unf	-5244.0305	5174.377	-1.013	0.311	-1.54e+0	
4	4907.758					
Heating_GasA	-1.115e+04	1.75e+04	-0.638	0.523	-4.54e+0	
4	2.31e+04					
Heating_GasW	-2.445e+04	1.8e+04	-1.357	0.175	-5.98e+0	
4	1.09e+04					
Heating_Grav	9325.7784	1.91e+04	0.487	0.626	-2.82e+0	
4	4.69e+04					
Heating_OthW	-3.582e+04	2.15e+04	-1.666	0.096	-7.8e+0	
4	6367.845					
Heating_Wall	-9452.3551	2.03e+04	-0.465	0.642	-4.93e+0	
4	3.04e+04					
HeatingQC_Fa	3325.6182	3219.784	1.033	0.302	-2991.38	
8	9642.625					
HeatingQC_Gd	-451.9679	1418.247	-0.319	0.750	-3234.47	
6	2330.541					
HeatingQC_Po	1.853e+04	1.81e+04	1.022	0.307	-1.7e+0	
4	5.41e+04					
HeatingQC_TA	2107.1534	1420.417	1.483	0.138	-679.61	
1	4893.918					
CentralAir_Y	-9999.4221	2653.244	-3.769	0.000	-1.52e+0	
4	-4793.932					
Electrical_FuseF	1297.8019	3925.370	0.331	0.741	-6403.51	
8	8999.122					
Electrical_FuseP	6557.4989	1.27e+04	0.516	0.606	-1.84e+0	
4	3.15e+04					
Electrical_Missing	-3065.2489	1.64e+04	-0.186	0.852	-3.53e+0	
4	2.92e+04					
Electrical_Mix	-3497.3004	3.04e+04	-0.115	0.908	-6.31e+0	
4	5.61e+04					
Electrical_SBrkr	323.0824	2014.417	0.160	0.873	-3629.07	
1	4275.236					
KitchenQual_Fa	-1.056e+04	4240.812	-2.490	0.013	-1.89e+0	
4	-2241.046					
KitchenQual_Gd	-1.301e+04	2392.966	-5.438	0.000	-1.77e+0	
4	-8318.768					
KitchenQual_TA	-1.187e+04	2697.309	-4.402	0.000	-1.72e+0	
4	-6581.567					
Functional_Maj2	3.962e+04	9871.574	4.013	0.000	2.03e+0	
4	5.9e+04					
Functional_Min1	1193.6952	5871.637	0.203	0.839	-1.03e+0	
4	1.27e+04					
Functional_Min2	5160.1403	5887.152	0.877	0.381	-6390.06	

7	1.67e+04					
Functional_Mod		5198.4217	7208.229	0.721	0.471	-8943.65
3	1.93e+04					
Functional_Sev		2411.0582	2.02e+04	0.119	0.905	-3.72e+0
4	4.21e+04					
Functional_Typ		8706.5898	5094.288	1.709	0.088	-1288.07
0	1.87e+04					
FireplaceQu_Fa		14.7866	4701.156	0.003	0.997	-9208.57
3	9238.147					
FireplaceQu_Gd		-195.5255	3632.419	-0.054	0.957	-7322.09
5	6931.044					
FireplaceQu_Missing		8606.8161	4251.144	2.025	0.043	266.35
0	1.69e+04					
FireplaceQu_Po		6215.9004	5404.490	1.150	0.250	-4387.35
6	1.68e+04					
FireplaceQu_TA		676.1279	3776.965	0.179	0.858	-6734.03
1	8086.286					
GarageType_Attchd		1892.0961	7540.210	0.251	0.802	-1.29e+0
4	1.67e+04					
GarageType_Basment		6126.1417	8740.976	0.701	0.484	-1.1e+0
4	2.33e+04					
GarageType_BuiltIn		4333.9074	7856.857	0.552	0.581	-1.11e+0
4	1.97e+04					
GarageType_CarPort		3493.5270	1e+04	0.348	0.728	-1.62e+0
4	2.32e+04					
GarageType_Detchd		5274.8371	7542.127	0.699	0.484	-9522.32
4	2.01e+04					
GarageType_Missing		4455.6553	3551.177	1.255	0.210	-2511.52
2	1.14e+04					
GarageFinish_Missing		4455.6553	3551.177	1.255	0.210	-2511.52
2	1.14e+04					
GarageFinish_RFn		-2686.6788	1338.745	-2.007	0.045	-5313.20
8	-60.149					
GarageFinish_Unf		987.0754	1658.577	0.595	0.552	-2266.94
4	4241.095					
GarageQual_Fa		-6.295e+04	2.07e+04	-3.048	0.002	-1.03e+0
5	-2.24e+04					
GarageQual_Gd		-6.754e+04	2.12e+04	-3.189	0.001	-1.09e+0
5	-2.6e+04					
GarageQual_Missing		4455.6553	3551.177	1.255	0.210	-2511.52
2	1.14e+04					
GarageQual_Po		-7.437e+04	2.63e+04	-2.828	0.005	-1.26e+0
5	-2.28e+04					
GarageQual_TA		-6.512e+04	2.04e+04	-3.187	0.001	-1.05e+0
5	-2.5e+04					
GarageCond_Fa		6.75e+04	2.38e+04	2.840	0.005	2.09e+0
4	1.14e+05					
GarageCond_Gd		6.196e+04	2.47e+04	2.508	0.012	1.35e+0
4	1.1e+05					
GarageCond_Missing		4455.6553	3551.177	1.255	0.210	-2511.52
2	1.14e+04					
GarageCond_Po		4.757e+04	2.55e+04	1.862	0.063	-2555.35
4	9.77e+04					
GarageCond_TA		6.57e+04	2.36e+04	2.787	0.005	1.95e+0
4	1.12e+05					
PavedDrive_P		-1517.7311	3790.297	-0.400	0.689	-8954.04

5	5918.583					
PavedDrive_Y	-2040.2080	2363.163	-0.863	0.388	-6676.58	
0	2596.164					
PoolQC_Fa	-1.349e+05	2.79e+04	-4.828	0.000	-1.9e+0	
5	-8.01e+04					
PoolQC_Gd	-1.323e+05	2.52e+04	-5.259	0.000	-1.82e+0	
5	-8.3e+04					
PoolQC_Missing	1.168e+05	8.38e+04	1.393	0.164	-4.77e+0	
4	2.81e+05					
Fence_GdWo	1.183e+04	3351.031	3.529	0.000	5250.85	
3	1.84e+04					
Fence_Missing	6593.7131	2507.133	2.630	0.009	1674.88	
2	1.15e+04					
Fence_MnPrv	8954.2774	2734.147	3.275	0.001	3590.06	
0	1.43e+04					
Fence_MnWw	4682.3766	5607.248	0.835	0.404	-6318.67	
8	1.57e+04					
MiscFeature_Missing	2.133e+04	6.64e+04	0.321	0.748	-1.09e+0	
5	1.52e+05					
MiscFeature_0thr	4.088e+04	6.2e+04	0.659	0.510	-8.08e+0	
4	1.63e+05					
MiscFeature_Shed	2.413e+04	6.36e+04	0.379	0.705	-1.01e+0	
5	1.49e+05					
MiscFeature_TenC	5.301e+04	6.6e+04	0.803	0.422	-7.65e+0	
4	1.83e+05					
SaleType_CWD	4952.1117	8787.005	0.564	0.573	-1.23e+0	
4	2.22e+04					
SaleType_Con	9922.1534	1.2e+04	0.828	0.408	-1.36e+0	
4	3.34e+04					
SaleType_ConLD	-4650.6961	6636.708	-0.701	0.484	-1.77e+0	
4	8370.090					
SaleType_ConLI	1.051e+04	7886.176	1.333	0.183	-4958.99	
6	2.6e+04					
SaleType_ConLw	-1328.7853	8293.778	-0.160	0.873	-1.76e+0	
4	1.49e+04					
SaleType_New	9485.9524	1.05e+04	0.901	0.368	-1.12e+0	
4	3.01e+04					
SaleType_0th	-3523.0483	9900.022	-0.356	0.722	-2.29e+0	
4	1.59e+04					
SaleType_WD	2368.4598	2852.882	0.830	0.407	-3228.70	
9	7965.629					
SaleCondition_AdjLand	-8665.8869	9980.670	-0.868	0.385	-2.82e+0	
4	1.09e+04					
SaleCondition_Alloca	-5657.9539	6057.984	-0.934	0.351	-1.75e+0	
4	6227.414					
SaleCondition_Family	-1359.2192	4157.049	-0.327	0.744	-9515.07	
7	6796.638					
SaleCondition_Normal	-3173.5951	2000.398	-1.586	0.113	-7098.24	
5	751.055					
SaleCondition_Partial	-2786.1660	1.01e+04	-0.275	0.783	-2.27e+0	
4	1.71e+04					

```

=====
==
Omnibus:              747.262    Durbin-Watson:              1.9
31
Prob(Omnibus):         0.000    Jarque-Bera (JB):              16476.4

```

```

19
Skew:                1.887    Prob(JB):                0.
00
Kurtosis:            19.019    Cond. No.                8.56e+
16
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 4.44e-23. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

```

In [46]: def backward_elimination(model, X, y, threshold=0.05):
        while True:
            # Fit the model and get p-values
            model = sm.OLS(y, sm.add_constant(X)).fit()
            pvalues = model.pvalues.iloc[1:]

            max_pval = pvalues.max()
            if max_pval > threshold:
                # Drop the feature with the highest p-value
                max_pval_feature = pvalues.idxmax()
                print(f"Dropping feature: {max_pval_feature} (p-value: {max_pval}")
                X = X.drop(columns=[max_pval_feature])
            else:
                break

            model = sm.OLS(y, sm.add_constant(X)).fit()
        return model, X

```

```

In [48]: # Perform backward elimination
        final_model, final_X = backward_elimination(full_model, X_full_encoded, y, t
        print(final_model.summary())

        print("Initial number of features:", X_full_encoded.shape[1])
        print("Final number of features:", final_X.shape[1])

```

Dropping feature: Utilities_NoSeWa (p-value: 0.9993962670459282)
Dropping feature: GrLivArea (p-value: 0.9988197627950618)
Dropping feature: FireplaceQu_Fa (p-value: 0.9974873225926599)
Dropping feature: Exterior2nd_Plywood (p-value: 0.9860995357282429)
Dropping feature: WoodDeckSF (p-value: 0.9807716372062725)
Dropping feature: OverallCond (p-value: 0.979388403418255)
Dropping feature: Condition1_PosN (p-value: 0.9727389798648545)
Dropping feature: Exterior2nd_AsphShn (p-value: 0.9364579188579625)
Dropping feature: FireplaceQu_Gd (p-value: 0.9328093897066365)
Dropping feature: Exterior2nd_Stucco (p-value: 0.9254892639777479)
Dropping feature: Alley_Missing (p-value: 0.9119675012684791)
Dropping feature: Electrical_Mix (p-value: 0.9096267420436962)
Dropping feature: Functional_Sev (p-value: 0.9059161677702702)
Dropping feature: Foundation_CBlock (p-value: 0.8767439103061152)
Dropping feature: Exterior2nd_Wd Sdng (p-value: 0.8810470173116094)
Dropping feature: SaleType_ConLw (p-value: 0.8745735828066131)
Dropping feature: HouseStyle_2.5Fin (p-value: 0.8737687361374922)
Dropping feature: Functional_Min1 (p-value: 0.8714246726877519)
Dropping feature: MSSubClass (p-value: 0.8616402949250963)
Dropping feature: Condition1_Feodr (p-value: 0.8641630887007513)
Dropping feature: Electrical_Missing (p-value: 0.857201947320696)
Dropping feature: Exterior1st_Stone (p-value: 0.851873098795632)
Dropping feature: Condition2_PosA (p-value: 0.8476157515882912)
Dropping feature: BsmtExposure_Missing (p-value: 0.8469199472036555)
Dropping feature: MiscVal (p-value: 0.8361820959554518)
Dropping feature: Electrical_SBrkr (p-value: 0.8350381646398759)
Dropping feature: Exterior2nd_Wd Shng (p-value: 0.827866978901713)
Dropping feature: Neighborhood_SawyerW (p-value: 0.8271863641345374)
Dropping feature: GarageType_Attchd (p-value: 0.7866779035627238)
Dropping feature: MasVnrType_Stone (p-value: 0.7868161045349394)
Dropping feature: SaleCondition_Partial (p-value: 0.772783570125174)
Dropping feature: Electrical_FuseF (p-value: 0.7641972672273288)
Dropping feature: LotConfig_FR3 (p-value: 0.7653483459875583)
Dropping feature: BsmtFinType1_LwQ (p-value: 0.7590407667497786)
Dropping feature: SaleCondition_Family (p-value: 0.749790864226134)
Dropping feature: GarageType_CarPort (p-value: 0.7481974570269762)
Dropping feature: BsmtFinType2_Missing (p-value: 0.744111145781882)
Dropping feature: BsmtFinType1_Missing (p-value: 0.8147573729737682)
Dropping feature: BsmtCond_Missing (p-value: 0.814757372973341)
Dropping feature: BsmtQual_Missing (p-value: 0.8147573729747994)
Dropping feature: LotShape_IR2 (p-value: 0.7420745668875942)
Dropping feature: Exterior2nd_Brk Cmn (p-value: 0.7400403133611881)
Dropping feature: HeatingQC_Gd (p-value: 0.7341384237212866)
Dropping feature: SaleType_0th (p-value: 0.7075536416651642)
Dropping feature: BsmtFinSF2 (p-value: 0.7061435918373291)
Dropping feature: LotShape_IR3 (p-value: 0.6989475930635918)
Dropping feature: Condition1_RRNN (p-value: 0.6971787339224533)
Dropping feature: PavedDrive_P (p-value: 0.6831772610341692)
Dropping feature: ExterCond_TA (p-value: 0.6788683863135138)
Dropping feature: LandContour_Lvl (p-value: 0.6803014647656129)
Dropping feature: Condition2_RRNN (p-value: 0.6954887223406799)
Dropping feature: MiscFeature_Missing (p-value: 0.676369556675573)
Dropping feature: ExterCond_Gd (p-value: 0.6972412510332815)
Dropping feature: GarageArea (p-value: 0.676901796950865)
Dropping feature: Exterior1st_CBlock (p-value: 0.6644929371933165)
Dropping feature: Exterior2nd_CBlock (p-value: 0.6644929371932946)

Dropping feature: OverallQual (p-value: 0.655137400674373)
Dropping feature: YrSold (p-value: 0.6496128799280929)
Dropping feature: Exterior2nd_Other (p-value: 0.6531745685178796)
Dropping feature: Foundation_Slab (p-value: 0.6338682993393143)
Dropping feature: BsmtFinType2_BLQ (p-value: 0.6466401995982947)
Dropping feature: BsmtFinType2_GLQ (p-value: 0.6701828375263231)
Dropping feature: Heating_Grav (p-value: 0.6519188461212303)
Dropping feature: FireplaceQu_TA (p-value: 0.6086217083411787)
Dropping feature: RoofStyle_Hip (p-value: 0.5916626202503139)
Dropping feature: RoofStyle_Mansard (p-value: 0.683380181761297)
Dropping feature: RoofStyle_Gable (p-value: 0.6613227637544852)
Dropping feature: BsmtUnfSF (p-value: 0.6113259423845188)
Dropping feature: Exterior2nd_HdBoard (p-value: 0.6002157077395234)
Dropping feature: RoofStyle_Gambrel (p-value: 0.5983816293002212)
Dropping feature: BsmtCond_TA (p-value: 0.6076690734670024)
Dropping feature: GarageYrBlt (p-value: 0.6063785986558843)
Dropping feature: Condition1_RRNe (p-value: 0.5944393565618404)
Dropping feature: OpenPorchSF (p-value: 0.5863541926251784)
Dropping feature: BsmtFinType1_Rec (p-value: 0.575038408211166)
Dropping feature: MasVnrType_Missing (p-value: 0.5760950491679222)
Dropping feature: GarageFinish_Unf (p-value: 0.5685709990791792)
Dropping feature: FullBath (p-value: 0.5841687712393347)
Dropping feature: Condition2_RRAn (p-value: 0.544529506865784)
Dropping feature: 3SsnPorch (p-value: 0.5407778614950725)
Dropping feature: LotConfig_FR2 (p-value: 0.5487231131190649)
Dropping feature: LotFrontage (p-value: 0.576861361083725)
Dropping feature: BsmtHalfBath (p-value: 0.5342118637241553)
Dropping feature: Exterior1st_BrkFace (p-value: 0.5286178886710742)
Dropping feature: LotShape_Reg (p-value: 0.5287988477623589)
Dropping feature: ExterCond_Po (p-value: 0.5297946129967117)
Dropping feature: Condition1_PosA (p-value: 0.5383726047380655)
Dropping feature: HouseStyle_SFoyer (p-value: 0.5033661994748093)
Dropping feature: Exterior2nd_MetalSd (p-value: 0.5027823352689147)
Dropping feature: SaleType_CWD (p-value: 0.4943070448150514)
Dropping feature: LandContour_HLS (p-value: 0.4823782676947739)
Dropping feature: Electrical_FuseP (p-value: 0.4424550424362722)
Dropping feature: SaleCondition_AdjLand (p-value: 0.5593896856073813)
Dropping feature: YearRemodAdd (p-value: 0.43062477640776764)
Dropping feature: SaleType_WD (p-value: 0.4529344726697879)
Dropping feature: SaleType_Con (p-value: 0.4942701207202441)
Dropping feature: Neighborhood_IDOTRR (p-value: 0.40996621689524004)
Dropping feature: Neighborhood_OldTown (p-value: 0.6479879731879715)
Dropping feature: Neighborhood_BrkSide (p-value: 0.7244150460089304)
Dropping feature: Neighborhood_Crawfor (p-value: 0.5003144960842938)
Dropping feature: Neighborhood_Veenker (p-value: 0.518247519431742)
Dropping feature: Neighborhood_Sawyer (p-value: 0.436215710115452)
Dropping feature: Neighborhood_Somerst (p-value: 0.47559062565909527)
Dropping feature: SaleCondition_Alloca (p-value: 0.43158279105564923)
Dropping feature: GarageType_Basment (p-value: 0.4710844031916749)
Dropping feature: Foundation_Wood (p-value: 0.43216912268102503)
Dropping feature: HeatingQC_Fa (p-value: 0.4016437487505259)
Dropping feature: Exterior1st_CemntBd (p-value: 0.4075413720519261)
Dropping feature: RoofStyle_Shed (p-value: 0.37888840343857766)
Dropping feature: Fence_MnWw (p-value: 0.3763197870402598)
Dropping feature: Foundation_Stone (p-value: 0.35326779072985415)
Dropping feature: HeatingQC_Po (p-value: 0.342045222895925)

Dropping feature: Neighborhood_SWISU (p-value: 0.33235830030450475)
Dropping feature: ScreenPorch (p-value: 0.328416279554396)
Dropping feature: MiscFeature_TenC (p-value: 0.35984289494831667)
Dropping feature: Functional_Mod (p-value: 0.30260755845599385)
Dropping feature: Functional_Min2 (p-value: 0.35355860412191265)
Dropping feature: Exterior2nd_VinylSd (p-value: 0.3204289304654727)
Dropping feature: Exterior2nd_BrkFace (p-value: 0.2966743869561718)
Dropping feature: GarageCars (p-value: 0.2745247572176196)
Dropping feature: Foundation_PConc (p-value: 0.2786120898576867)
Dropping feature: GarageType_BuiltIn (p-value: 0.271062613651168)
Dropping feature: SaleType_ConLD (p-value: 0.2708343267563143)
Dropping feature: PavedDrive_Y (p-value: 0.26273029951106774)
Dropping feature: Alley_Pave (p-value: 0.2868338076595009)
Dropping feature: Neighborhood_Edwards (p-value: 0.26717830538395076)
Dropping feature: Condition2_RRAe (p-value: 0.2561262659563113)
Dropping feature: HouseStyle_2Story (p-value: 0.2856163327471781)
Dropping feature: PoolQC_Missing (p-value: 0.24143107418238297)
Dropping feature: Condition1_RRAn (p-value: 0.27699949122723566)
Dropping feature: Condition1_Norm (p-value: 0.2967154101072313)
Dropping feature: Neighborhood_CollgCr (p-value: 0.22670362439495073)
Dropping feature: BsmtCond_Po (p-value: 0.2233920286580631)
Dropping feature: LandSlope_Sev (p-value: 0.22059299276387817)
Dropping feature: SaleType_ConLI (p-value: 0.1999205968173511)
Dropping feature: Condition1_RRAe (p-value: 0.20914785145954204)
Dropping feature: Neighborhood_Gilbert (p-value: 0.19893220337052162)
Dropping feature: YearBuilt (p-value: 0.24179786734753295)
Dropping feature: Exterior1st_AsphShn (p-value: 0.20394928153366387)
Dropping feature: GarageCond_Missing (p-value: 0.2085598815964327)
Dropping feature: GarageType_Missing (p-value: 0.20855988159539204)
Dropping feature: GarageFinish_Missing (p-value: 0.20855988159959576)
Dropping feature: GarageQual_Missing (p-value: 0.2085598815956379)
Dropping feature: SaleCondition_Normal (p-value: 0.17636640597600967)
Dropping feature: Exterior2nd_Stone (p-value: 0.1893657833188669)
Dropping feature: Neighborhood_Timber (p-value: 0.18894689937309372)
Dropping feature: BsmtExposure_Mn (p-value: 0.16325066534023744)
Dropping feature: Exterior1st_ImStucc (p-value: 0.15733547689201552)
Dropping feature: Exterior2nd_ImStucc (p-value: 0.13474687794612683)
Dropping feature: BsmtFinType1_BLQ (p-value: 0.12446439590625565)
Dropping feature: MiscFeature_Shed (p-value: 0.13168474739203254)
Dropping feature: BsmtFinType2_Rec (p-value: 0.12132166304334051)
Dropping feature: BsmtFinType2_Unf (p-value: 0.18658420798612696)
Dropping feature: BsmtFinType2_LwQ (p-value: 0.21533579620634977)
Dropping feature: BsmtFinType1_GLQ (p-value: 0.1735006068955876)
Dropping feature: LandSlope_Mod (p-value: 0.17515727015953453)
Dropping feature: BsmtFinType1_Unf (p-value: 0.16336827864254747)
Dropping feature: GarageCond_Po (p-value: 0.122093145665774)
Dropping feature: Neighborhood_Blueste (p-value: 0.11988919573137632)
Dropping feature: FireplaceQu_Po (p-value: 0.11326108748553185)
Dropping feature: Id (p-value: 0.12079560029808548)
Dropping feature: Condition2_Norm (p-value: 0.10803157784908458)
Dropping feature: Condition2_Feodr (p-value: 0.19406269555313718)
Dropping feature: BsmtFullBath (p-value: 0.10455076173327556)
Dropping feature: MoSold (p-value: 0.08857278080481722)
Dropping feature: Heating_Wall (p-value: 0.07149802915627568)
Dropping feature: MiscFeature_Othr (p-value: 0.07021820811060239)
Dropping feature: ExterQual_Fa (p-value: 0.08605287486162878)

Dropping feature: HouseStyle_SLvl (p-value: 0.0794051707560797)
Dropping feature: BsmtCond_Gd (p-value: 0.08439505140898683)
Dropping feature: GarageType_Detchd (p-value: 0.07449095319121624)
Dropping feature: KitchenAbvGr (p-value: 0.058236039587572425)
Dropping feature: Exterior1st_Wd Sdng (p-value: 0.0596519760921987)
Dropping feature: Exterior1st_WdShing (p-value: 0.20800030638382902)
Dropping feature: Exterior2nd_CmentBd (p-value: 0.1899138912853644)
Dropping feature: Exterior1st_MetalSd (p-value: 0.23221260587845985)
Dropping feature: Exterior1st_HdBoard (p-value: 0.39034587481859306)
Dropping feature: Exterior1st_VinylSd (p-value: 0.23297486278782806)
Dropping feature: Exterior1st_Stucco (p-value: 0.1486684402048842)
Dropping feature: Exterior1st_Plywood (p-value: 0.14475849542143923)
Dropping feature: TotRmsAbvGrd (p-value: 0.0820994009900846)
Dropping feature: HeatingQC_TA (p-value: 0.09010361963264572)

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice    R-squared:                0.9
65
Model:                  OLS          Adj. R-squared:            0.9
63
Method:                 Least Squares    F-statistic:              47
7.2
Date:                   Sun, 22 Dec 2024    Prob (F-statistic):        0.
00
Time:                   22:34:56          Log-Likelihood:            -1609
4.
No. Observations:      1460              AIC:                      3.235e+
04
Df Residuals:          1379              BIC:                      3.278e+
04
Df Model:               80
Covariance Type:       nonrobust
=====
```

```
=====
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const          -1.842e+06    3.33e+04    -55.283    0.000    -1.91e+06
-1.78e+06
LotArea         0.2033         0.052         3.926    0.000         0.102
0.305
MasVnrArea     18.1967         3.353         5.427    0.000        11.620
24.774
BsmtFinSF1      4.0104         1.205         3.327    0.001         1.646
6.375
TotalBsmtSF     6.5559         2.141         3.062    0.002         2.356
10.756
1stFlrSF       12.0298         2.652         4.537    0.000         6.828
17.231
2ndFlrSF       25.2268         2.401        10.505    0.000        20.516
29.938
LowQualFinSF   -33.5242         9.085        -3.690    0.000        -51.346
-15.703
HalfBath      -2608.3171    1127.328        -2.314    0.021    -4819.781
=====
```


-396.853					
BedroomAbvGr	-4160.7879	735.735	-5.655	0.000	-5604.069
-2717.507					
Fireplaces	4167.1560	1598.252	2.607	0.009	1031.888
7302.424					
EnclosedPorch	-21.4668	7.255	-2.959	0.003	-35.698
-7.236					
PoolArea	212.7099	21.315	9.979	0.000	170.897
254.523					
LogSalePrice	1.57e+05	3054.577	51.401	0.000	1.51e+05
1.63e+05					
LotArea_scaled	0.0002	5.18e-05	3.925	0.000	0.000
0.000					
MSZoning_FV	-3.564e+04	6009.124	-5.931	0.000	-4.74e+04
-2.39e+04					
MSZoning_RH	-4.401e+04	6665.576	-6.602	0.000	-5.71e+04
-3.09e+04					
MSZoning_RL	-3.988e+04	5556.247	-7.178	0.000	-5.08e+04
-2.9e+04					
MSZoning_RM	-3.624e+04	5497.971	-6.592	0.000	-4.7e+04
-2.55e+04					
Street_Pave	2.006e+04	6920.329	2.899	0.004	6485.438
3.36e+04					
LandContour_Low	-7904.1863	3167.129	-2.496	0.013	-1.41e+04
-1691.275					
LotConfig_CulDSac	4642.3564	1909.670	2.431	0.015	896.183
8388.530					
LotConfig_Inside	2035.4998	1011.319	2.013	0.044	51.610
4019.389					
Neighborhood_BrDale	1.713e+04	4835.479	3.543	0.000	7645.861
2.66e+04					
Neighborhood_ClearCr	-1.178e+04	3443.809	-3.421	0.001	-1.85e+04
-5025.350					
Neighborhood_MeadowV	2.306e+04	4392.339	5.250	0.000	1.44e+04
3.17e+04					
Neighborhood_Mitchel	-5866.9785	2386.375	-2.459	0.014	-1.05e+04
-1185.660					
Neighborhood_NAmes	-4065.0024	1386.703	-2.931	0.003	-6785.277
-1344.728					
Neighborhood_NPkVill	1.338e+04	5533.057	2.419	0.016	2529.155
2.42e+04					
Neighborhood_NWAmes	-4151.0394	2055.114	-2.020	0.044	-8182.528
-119.551					
Neighborhood_NoRidge	2.959e+04	2843.578	10.404	0.000	2.4e+04
3.52e+04					
Neighborhood_NridgHt	1.273e+04	2384.028	5.341	0.000	8057.289
1.74e+04					
Neighborhood_StoneBr	2.468e+04	3422.106	7.212	0.000	1.8e+04
3.14e+04					
Condition2_PosN	-9.148e+04	1.17e+04	-7.793	0.000	-1.15e+05
-6.85e+04					
BldgType_2fmCon	-1.452e+04	3078.859	-4.715	0.000	-2.06e+04
-8477.986					
BldgType_Duplex	-1.151e+04	2567.372	-4.482	0.000	-1.65e+04
-6470.235					
BldgType_Twnhs	-1.036e+04	3185.216	-3.251	0.001	-1.66e+04

-4107.470					
BldgType_TwnhsE	-1.188e+04	1956.486	-6.072	0.000	-1.57e+04
-8042.246					
HouseStyle_1.5Unf	1.146e+04	4515.078	2.538	0.011	2602.864
2.03e+04					
HouseStyle_1Story	6705.0111	1595.841	4.202	0.000	3574.472
9835.550					
HouseStyle_2.5Unf	-1.186e+04	5066.752	-2.340	0.019	-2.18e+04
-1917.715					
RoofMatl_CompShg	1.966e+05	2.39e+04	8.233	0.000	1.5e+05
2.43e+05					
RoofMatl_Membran	2.05e+05	2.92e+04	7.025	0.000	1.48e+05
2.62e+05					
RoofMatl_Metal	1.913e+05	2.89e+04	6.619	0.000	1.35e+05
2.48e+05					
RoofMatl_Roll	1.869e+05	2.86e+04	6.542	0.000	1.31e+05
2.43e+05					
RoofMatl_Tar&Grv	1.863e+05	2.4e+04	7.774	0.000	1.39e+05
2.33e+05					
RoofMatl_WdShake	2.013e+05	2.48e+04	8.116	0.000	1.53e+05
2.5e+05					
RoofMatl_WdShngl	2.465e+05	2.45e+04	10.076	0.000	1.99e+05
2.95e+05					
Exterior1st_BrkComm	3.726e+04	1.17e+04	3.184	0.001	1.43e+04
6.02e+04					
MasVnrType_BrkFace	-3736.6740	1171.101	-3.191	0.001	-6034.006
-1439.342					
ExterQual_Gd	-1.979e+04	2591.114	-7.638	0.000	-2.49e+04
-1.47e+04					
ExterQual_TA	-1.991e+04	2735.606	-7.279	0.000	-2.53e+04
-1.45e+04					
ExterCond_Fa	8527.3568	3334.891	2.557	0.011	1985.348
1.51e+04					
BsmtQual_Fa	-9800.6054	3359.792	-2.917	0.004	-1.64e+04
-3209.749					
BsmtQual_Gd	-1.44e+04	1830.033	-7.869	0.000	-1.8e+04
-1.08e+04					
BsmtQual_TA	-1.131e+04	2025.287	-5.585	0.000	-1.53e+04
-7337.245					
BsmtExposure_Gd	1.041e+04	1766.151	5.892	0.000	6940.734
1.39e+04					
BsmtExposure_No	-3432.7270	1061.371	-3.234	0.001	-5514.804
-1350.650					
Heating_GasA	-1.825e+04	5043.609	-3.619	0.000	-2.81e+04
-8358.025					
Heating_GasW	-3.123e+04	6242.343	-5.003	0.000	-4.35e+04
-1.9e+04					
Heating_0thW	-4.058e+04	1.23e+04	-3.303	0.001	-6.47e+04
-1.65e+04					
CentralAir_Y	-1.058e+04	2195.096	-4.820	0.000	-1.49e+04
-6274.786					
KitchenQual_Fa	-1.287e+04	3559.309	-3.617	0.000	-1.99e+04
-5891.937					
KitchenQual_Gd	-1.521e+04	2156.530	-7.052	0.000	-1.94e+04
-1.1e+04					
KitchenQual_TA	-1.369e+04	2389.361	-5.731	0.000	-1.84e+04

-9006.525					
Functional_Maj2	3.639e+04	7578.980	4.802	0.000	2.15e+04
5.13e+04					
Functional_Typ	5938.1118	1848.917	3.212	0.001	2311.118
9565.105					
FireplaceQu_Missing	7982.4692	2033.300	3.926	0.000	3993.773
1.2e+04					
GarageFinish_RFn	-3716.0634	990.382	-3.752	0.000	-5658.882
-1773.245					
GarageQual_Fa	-3.461e+04	7479.499	-4.627	0.000	-4.93e+04
-1.99e+04					
GarageQual_Gd	-3.605e+04	9134.512	-3.947	0.000	-5.4e+04
-1.81e+04					
GarageQual_Po	-2.849e+04	9880.084	-2.884	0.004	-4.79e+04
-9112.801					
GarageQual_TA	-3.875e+04	7800.206	-4.968	0.000	-5.41e+04
-2.35e+04					
GarageCond_Fa	2.871e+04	7787.906	3.687	0.000	1.34e+04
4.4e+04					
GarageCond_Gd	2.209e+04	9632.529	2.294	0.022	3198.250
4.1e+04					
GarageCond_TA	2.683e+04	7677.373	3.494	0.000	1.18e+04
4.19e+04					
PoolQC_Fa	-1.18e+05	1.66e+04	-7.125	0.000	-1.51e+05
-8.55e+04					
PoolQC_Gd	-1.168e+05	1.81e+04	-6.442	0.000	-1.52e+05
-8.12e+04					
Fence_GdWo	1.085e+04	2875.471	3.773	0.000	5209.603
1.65e+04					
Fence_Missing	4407.6026	2011.374	2.191	0.029	461.918
8353.287					
Fence_MnPrv	7475.8480	2291.165	3.263	0.001	2981.302
1.2e+04					
SaleType_New	6912.8506	1735.053	3.984	0.000	3509.223
1.03e+04					

```

=====
==
Omnibus:                732.856    Durbin-Watson:                1.9
62
Prob(Omnibus):           0.000    Jarque-Bera (JB):            15872.2
19
Skew:                    1.842    Prob(JB):                     0.
00
Kurtosis:                18.727    Cond. No.                    1.37e+
16
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.65e-21. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Initial number of features: 264

Final number of features: 82

We can even go further and apply Recursive Feature Elimination (ranks features by importance and selects the optimal subset) and Lasso Regression (introduces a penalty for the absolute size of coefficients, shrinks features with small coefficients to zero and removes them).

```
In [55]: # Recursive Feature Elimination (RFE)
linear_model = LinearRegression()
rfe = RFE(estimator=linear_model, n_features_to_select=10) # Top 10 features
rfe.fit(final_X, y)
rfe_features = final_X.columns[rfe.support_]
print("Selected Features with RFE:", rfe_features)

# Lasso with cross-validation to determine the optimal alpha
lasso = LassoCV(cv=5).fit(final_X, y)
lasso_features = final_X.columns[lasso.coef_ != 0]
print("Selected Features with Lasso:", lasso_features)

# Combine RFE and Lasso features and retrain the model
combined_features = set(lasso_features).union(set(rfe_features))
reduced_X = final_X[list(combined_features)]

reduced_model = sm.OLS(y, sm.add_constant(reduced_X)).fit()
print(reduced_model.summary())
```

```
Selected Features with RFE: Index(['LogSalePrice', 'RoofMatl_CompShg', 'Roof
Matl_Membran',
    'RoofMatl_Metal', 'RoofMatl_Roll', 'RoofMatl_Tar&Grv',
    'RoofMatl_WdShake', 'RoofMatl_WdShngl', 'Exterior1st_BrkComm',
    'ExterQual_TA'],
    dtype='object')
Selected Features with Lasso: Index(['LotArea', 'MasVnrArea', 'BsmtFinSF1',
    'TotalBsmtSF', '1stFlrSF',
    '2ndFlrSF', 'EnclosedPorch'],
    dtype='object')
```

OLS Regression Results

=====			
==			
Dep. Variable:	SalePrice	R-squared:	0.9
17			
Model:	OLS	Adj. R-squared:	0.9
16			
Method:	Least Squares	F-statistic:	93
6.7			
Date:	Sun, 22 Dec 2024	Prob (F-statistic):	0.
00			
Time:	22:44:52	Log-Likelihood:	-1672
7.			
No. Observations:	1460	AIC:	3.349e+
04			
Df Residuals:	1442	BIC:	3.359e+
04			
Df Model:	17		
Covariance Type:	nonrobust		
=====			

	coef	std err	t	P> t	[0.025
0.975]					

const	-1.873e+06	3.49e+04	-53.733	0.000	-1.94e+06
-1.81e+06					
RoofMatl_WdShake	1.696e+05	2.91e+04	5.819	0.000	1.12e+05
2.27e+05					
LogSalePrice	1.527e+05	3060.249	49.904	0.000	1.47e+05
1.59e+05					
RoofMatl_WdShngl	2.496e+05	2.86e+04	8.735	0.000	1.94e+05
3.06e+05					
ExterQual_TA	-9310.5847	1612.657	-5.773	0.000	-1.25e+04
-6147.180					
MasVnrArea	30.7441	3.819	8.050	0.000	23.253
38.235					
LotArea	0.1078	0.066	1.643	0.101	-0.021
0.237					
TotalBsmtSF	4.4814	2.779	1.613	0.107	-0.970
9.932					
RoofMatl_Tar&Grv	1.764e+05	2.82e+04	6.259	0.000	1.21e+05
2.32e+05					
Exterior1st_BrkComm	4.525e+04	1.68e+04	2.698	0.007	1.24e+04
7.81e+04					
RoofMatl_CompShg	1.849e+05	2.73e+04	6.772	0.000	1.31e+05

2.39e+05					
EnclosedPorch	12.9041	10.174	1.268	0.205	-7.054
32.862					
1stFlrSF	17.8807	3.044	5.873	0.000	11.909
23.853					
RoofMatl_Roll	1.578e+05	3.55e+04	4.443	0.000	8.81e+04
2.27e+05					
BsmtFinSF1	8.3298	1.653	5.039	0.000	5.087
11.573					
RoofMatl_Metal	1.828e+05	3.59e+04	5.085	0.000	1.12e+05
2.53e+05					
RoofMatl_Membran	1.947e+05	3.61e+04	5.396	0.000	1.24e+05
2.66e+05					
2ndFlrSF	14.5352	1.921	7.567	0.000	10.767
18.303					
=====					
==					
Omnibus:	1173.047	Durbin-Watson:		1.9	
22					
Prob(Omnibus):	0.000	Jarque-Bera (JB):		42477.4	
26					
Skew:	3.456	Prob(JB):		0.	
00					
Kurtosis:	28.505	Cond. No.		1.82e+	
06					
=====					
==					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Explanation of model selection methods

- **AIC** considers model fit and parameter count
- **RFE** explicitly ranks and removes features iteratively
- **Lasso** penalizes feature coefficients, automatically selecting the most relevant ones

1. Akaike Information Criterion (AIC): evaluate the goodness-of-fit of a model while penalizing for model complexity to prevent overfitting; during selection, models are compared, and the one with the lowest AIC is preferred:

$$AIC = -2 \cdot \text{Log-Likelihood} + 2 \cdot k$$

- Log-Likelihood: measures how well the model explains the observed data
- k : number of parameters

2. Recursive Feature Elimination (RFE): iteratively removes the least significant features based on model performance. It uses model weights or importance scores to assess feature relevance. At each step, the feature with the lowest importance score is eliminated.
3. Lasso: performs both variable selection and regularization; adds an L_1 -penalty to the ordinary least squares objective, shrinks some coefficients to zero, removing less important features, by tuning λ (often through cross-validation):

$$\text{Minimize: } \frac{1}{2n} \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p |\beta_j|$$

- n : number of observations
- p : number of features
- λ : regularization parameter controlling the penalty
- β_j : coefficients of the features

Task 1.8

Sometimes data contains outliers which induce bias in the parameter estimates. Check for outliers using plots of the residuals, Cook's distance, and leverage. Have a closer look at the observation with the highest leverage (regardless of whether it is classified as an outlier or not). What makes this observation so outstanding? (You may examine Box-plots for interval-scaled variables or look at the frequencies for binary/ordinal variables.)

```
In [79]: # Residuals and leverage
influence = reduced_model.get_influence()
leverage = influence.hat_matrix_diag
cooks_d = influence.cooks_distance[0]
standardized_residuals = influence.resid_studentized_internal

# Leverage vs. Standardized Residuals
plt.figure(figsize=(10, 6))
plt.scatter(leverage, standardized_residuals, alpha=0.5)
plt.axhline(y=0, color="red", linestyle="--", linewidth=1) # Reference line
plt.axvline(x=0.2, color="green", linestyle="--", label="Potential high leve
plt.xlabel("Leverage")
plt.ylabel("Standardized Residuals")
plt.title("Leverage vs. Standardized Residuals")
plt.legend()
plt.grid()
plt.show()

# 2. Cook's Distance for each observation
plt.figure(figsize=(10, 6))
plt.stem(np.arange(len(cooks_d)), cooks_d, markerfmt=","")
```

```

plt.axhline(y=4 / len(cooks_d), color="red", linestyle="--", label="Threshold")
plt.title("Cook's Distance for Each Observation")
plt.xlabel("Observation Index")
plt.ylabel("Cook's Distance")
plt.legend()
plt.grid()
plt.show()

max_leverage_idx = np.argmax(leverage)
print(f"Observation with the highest leverage: {max_leverage_idx}")
print(f"Leverage for the observation: {leverage[max_leverage_idx]}")
print(f"Cook's Distance for the observation: {cooks_d[max_leverage_idx]}")

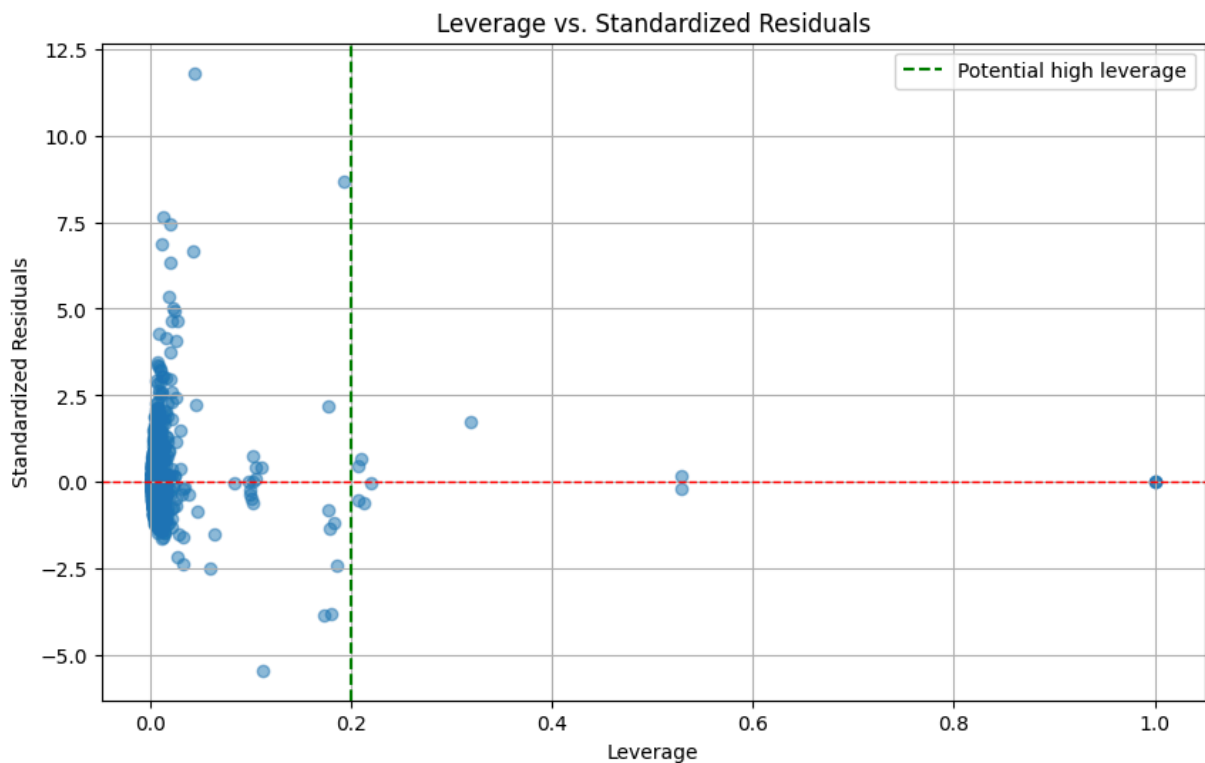
print("\nDetails of the observation with the highest leverage:")
print(reduced_X.iloc[max_leverage_idx])
print("\nTarget value for the observation:")
print(y.iloc[max_leverage_idx])

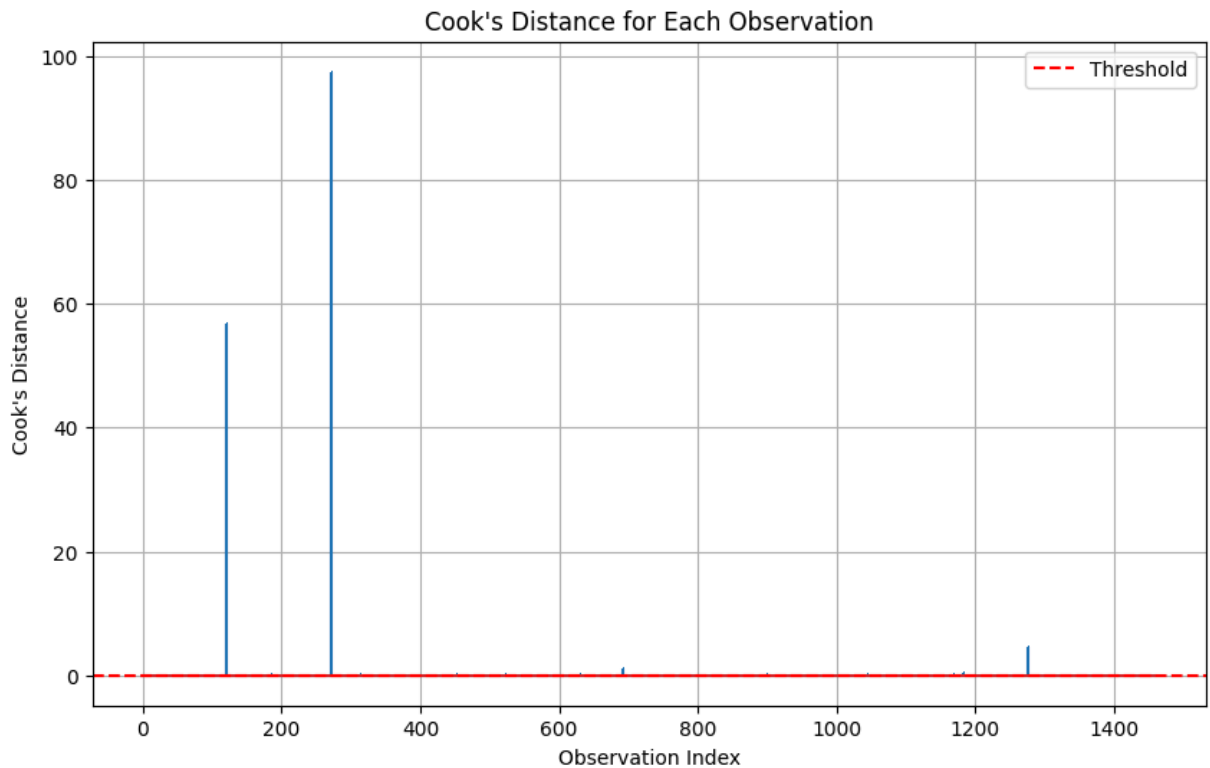
```

```

/usr/local/lib/python3.10/dist-packages/statsmodels/stats/outliers_influence.py:847: RuntimeWarning: invalid value encountered in sqrt
    return self.resid / sigma / np.sqrt(1 - hii)

```





Observation with the highest leverage: 1298
 Leverage for the observation: 1.0000000000066585
 Cook's Distance for the observation: nan

Details of the observation with the highest leverage:

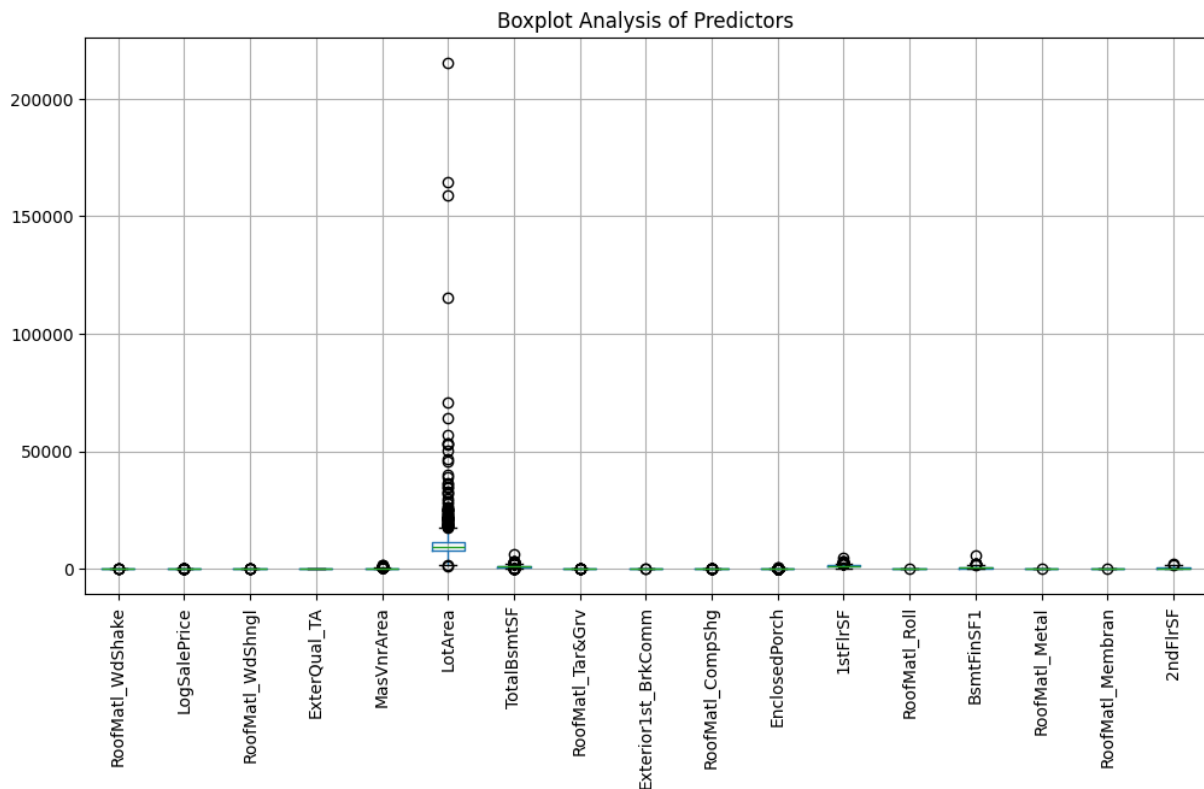
RoofMatl_WdShake	0.000000
LogSalePrice	11.982929
RoofMatl_WdShngl	0.000000
ExterQual_TA	0.000000
MasVnrArea	796.000000
LotArea	63887.000000
TotalBsmtSF	6110.000000
RoofMatl_Tar&Grv	0.000000
Exterior1st_BrkComm	0.000000
RoofMatl_CompShg	0.000000
EnclosedPorch	0.000000
1stFlrSF	4692.000000
RoofMatl_Roll	0.000000
BsmtFinSF1	5644.000000
RoofMatl_Metal	0.000000
RoofMatl_Membran	0.000000
2ndFlrSF	950.000000

Name: 1298, dtype: float64

Target value for the observation:
 160000

The observation 1298 has extremely high leverage, indicating it strongly influences the regression. We even cannot estimate the residual at thi point. It is a strong outlier and could distort the model's performance if not addressed.

```
In [80]: # Boxplot analysis of predictors
plt.figure(figsize=(12, 6))
reduced_X.boxplot()
plt.title("Boxplot Analysis of Predictors")
plt.xticks(rotation=90)
plt.show()
```



```
In [83]: # Remove the most influential point (highest leverage) and refit the model
reduced_X_no_outlier = reduced_X.drop(index=max_leverage_idx)
y_no_outlier = y.drop(index=max_leverage_idx)

reduced_model_no_outlier = sm.OLS(y_no_outlier, sm.add_constant(reduced_X_no_outlier))
print("\nModel Summary after removing the most influential point:")
print(reduced_model_no_outlier.summary())

# Observations with high Cook's Distance
threshold = 4 / len(cooks_d)
high_cooks = np.where(cooks_d > threshold)[0]
print(f"\nObservations with high Cook's Distance (Threshold: {threshold}): {high_cooks}")
```

Model Summary after removing the most influential point:

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice  R-squared:                0.9
17
Model:                  OLS      Adj. R-squared:            0.9
16
Method:                 Least Squares  F-statistic:           99
5.1
Date:                   Mon, 23 Dec 2024  Prob (F-statistic):      0.
00
Time:                   00:38:18   Log-Likelihood:         -1671
7.
No. Observations:      1459      AIC:                   3.347e+
04
Df Residuals:          1442      BIC:                   3.356e+
04
Df Model:               16
Covariance Type:       nonrobust
=====
```

```
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -1.475e+06    3.11e+04    -47.452    0.000    -1.54e+06
-1.41e+06
RoofMatl_WdShake     -2.291e+05    1.13e+04    -20.346    0.000    -2.51e+05
-2.07e+05
LogSalePrice         1.527e+05    3060.249     49.904    0.000     1.47e+05
1.59e+05
RoofMatl_WdShngl     -1.491e+05    1.07e+04    -13.879    0.000    -1.7e+05
-1.28e+05
ExterQual_TA        -9310.5847    1612.657     -5.773    0.000    -1.25e+04
-6147.180
MasVnrArea           30.7441         3.819        8.050    0.000        23.253
38.235
LotArea              0.1078         0.066        1.643    0.101        -0.021
0.237
TotalBsmtSF          4.4814         2.779        1.613    0.107        -0.970
9.932
RoofMatl_Tar&Grv     -2.222e+05    9236.336    -24.061    0.000    -2.4e+05
-2.04e+05
Exterior1st_BrkComm  4.525e+04    1.68e+04      2.698    0.007     1.24e+04
7.81e+04
RoofMatl_CompShg     -2.137e+05    6825.656    -31.313    0.000    -2.27e+05
-2e+05
EnclosedPorch        12.9041        10.174        1.268    0.205        -7.054
32.862
1stFlrSF             17.8807         3.044        5.873    0.000        11.909
23.853
RoofMatl_Roll        -2.409e+05    2.1e+04    -11.460    0.000    -2.82e+05
-2e+05
BsmtFinSF1           8.3298         1.653        5.039    0.000         5.087
11.573
=====
```

RoofMatl_Metal	-2.159e+05	2.13e+04	-10.141	0.000	-2.58e+05
-1.74e+05					
RoofMatl_Membran	-2.039e+05	2.14e+04	-9.522	0.000	-2.46e+05
-1.62e+05					
2ndFlrSF	14.5352	1.921	7.567	0.000	10.767
18.303					

```

==
Omnibus:                1171.914    Durbin-Watson:                1.9
22
Prob(Omnibus):           0.000    Jarque-Bera (JB):            42385.8
23
Skew:                    3.454    Prob(JB):                     0.
00
Kurtosis:                28.485    Cond. No.                     4.51e+
18
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 1.5e-26. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Observations with high Cook's Distance (Threshold: 0.0027397260273972603): [

```

30  53  58  70 115 120 169 178 185 197 231 249 271 278
   297 313 324 335 349 375 389 440 451 462 471 473 495 496
   523 527 529 533 591 608 629 636 664 688 691 705 710 769
   774 798 803 808 812 898 916 968 1044 1046 1100 1142 1169 1181
  1182 1243 1267 1268 1270 1275 1325 1337 1373 1380 1423]

```

```

In [82]: # Comparing the before and after removing observations
if len(high_cooks) > 0:
    reduced_X_no_cooks = reduced_X.drop(index=high_cooks)
    y_no_cooks = y.drop(index=high_cooks)
    reduced_model_no_cooks = sm.OLS(y_no_cooks, sm.add_constant(reduced_X_no_cooks))
    print("\nModel summary after removing influential points:")
    print(reduced_model_no_cooks.summary())

    print("\nDetails of influential points removed:")
    print(reduced_X.iloc[high_cooks])
    print("\nTarget values for Influential Points Removed:")
    print(y.iloc[high_cooks])
else:
    print("No observations exceed the Cook's Distance threshold.")

```

Model Summary after Removing Influential Points:
OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice  R-squared:                0.9
57
Model:                  OLS      Adj. R-squared:              0.9
57
Method:                 Least Squares  F-statistic:            237
0.
Date:                  Mon, 23 Dec 2024  Prob (F-statistic):        0.
00
Time:                  00:37:27   Log-Likelihood:            -1520
7.
No. Observations:      1393      AIC:                      3.044e+
04
Df Residuals:          1379      BIC:                      3.051e+
04
Df Model:              13
Covariance Type:       nonrobust
=====
```

```
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -1.843e+06    2.26e+04    -81.386    0.000    -1.89e+06
-1.8e+06
RoofMatl_WdShake      1.032e+05    2.14e+04     4.822    0.000     6.12e+04
1.45e+05
LogSalePrice          1.583e+05    2150.495    73.608    0.000     1.54e+05
1.63e+05
RoofMatl_WdShngl     -1.361e-06    3.82e-08   -35.600    0.000    -1.44e-06
-1.29e-06
ExterQual_TA         -7292.2323    1021.858     -7.136    0.000    -9296.797
-5287.668
MasVnrArea            22.8101        2.605        8.758    0.000        17.701
27.920
LotArea               0.1370         0.073        1.885    0.060        -0.006
0.280
TotalBsmtSF           0.6882         1.685         0.408    0.683        -2.617
3.993
RoofMatl_Tar&Grv      9.586e+04    1.73e+04     5.544    0.000     6.19e+04
1.3e+05
Exterior1st_BrkComm   4.355e+04    9800.797     4.443    0.000     2.43e+04
6.28e+04
RoofMatl_CompShg      1.024e+05    1.69e+04     6.058    0.000     6.92e+04
1.36e+05
EnclosedPorch         11.1167         6.240         1.782    0.075        -1.123
23.357
1stFlrSF              9.2272         1.919         4.808    0.000         5.463
12.992
RoofMatl_Roll        -1.465e-11    1.39e-12   -10.537    0.000    -1.74e-11
-1.19e-11
BsmtFinSF1            4.5907         1.022         4.493    0.000         2.586
6.595
=====
```

RoofMatl_Metal	0	0	nan	nan	0
0					
RoofMatl_Membran	0	0	nan	nan	0
0					
2ndFlrSF	5.8409	1.235	4.731	0.000	3.419
8.263					

=====

==

Omnibus:	441.904	Durbin-Watson:	2.0
11			
Prob(Omnibus):	0.000	Jarque-Bera (JB):	1293.6
45			
Skew:	1.620	Prob(JB):	1.23e-2
81			
Kurtosis:	6.433	Cond. No.	i
nf			

=====

==

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The smallest eigenvalue is 0. This might indicate that there are strong multicollinearity problems or that the design matrix is singular.

Details of Influential Points Removed:

	RoofMatl_WdShake	LogSalePrice	RoofMatl_WdShngl	ExterQual_TA	\
30	0.0	10.596635	0.0	1.0	
53	0.0	12.860999	1.0	0.0	
58	0.0	12.991753	0.0	0.0	
70	0.0	12.404924	0.0	1.0	
115	0.0	12.078239	0.0	1.0	
...	...	...	...	...	
1325	0.0	10.915088	0.0	1.0	
1337	0.0	10.868568	0.0	1.0	
1373	0.0	13.053013	0.0	0.0	
1380	0.0	10.976782	0.0	1.0	
1423	0.0	12.524417	0.0	0.0	

	MasVnrArea	LotArea	TotalBsmtSF	RoofMatl_Tar&Grv	Exterior1st_BrkCom
m \					
30	0.0	8500.0	649.0	0.0	0.
0					
53	0.0	50271.0	1842.0	0.0	0.
0					
58	1031.0	13682.0	1410.0	0.0	0.
0					
70	1115.0	13651.0	2223.0	0.0	0.
0					
115	1129.0	3230.0	729.0	0.0	0.
0					
...	...	...	...	...	
...					
1325	0.0	3636.0	796.0	0.0	0.
0					
1337	0.0	4118.0	693.0	0.0	0.

0					
1373	705.0	11400.0	2633.0	0.0	0.
0					
1380	0.0	8212.0	864.0	0.0	0.
0					
1423	0.0	19690.0	697.0	1.0	0.
0					

	RoofMatl_CompShg	EnclosedPorch	1stFlrSF	RoofMatl_Roll	BsmtFinSF1
\					
30	1.0	172.0	649.0	0.0	0.0
53	0.0	0.0	1842.0	0.0	1810.0
58	1.0	37.0	1426.0	0.0	0.0
70	1.0	0.0	2223.0	0.0	1880.0
115	1.0	0.0	729.0	0.0	419.0
...	...	...	...	...	...
1325	1.0	100.0	796.0	0.0	0.0
1337	1.0	0.0	693.0	0.0	0.0
1373	1.0	0.0	2633.0	0.0	1282.0
1380	1.0	96.0	864.0	0.0	203.0
1423	0.0	0.0	1575.0	0.0	0.0

	RoofMatl_Metal	RoofMatl_Membran	2ndFlrSF
30	0.0	0.0	668.0
53	0.0	0.0	0.0
58	0.0	0.0	1519.0
70	0.0	0.0	0.0
115	0.0	0.0	729.0
...	...	...	...
1325	0.0	0.0	0.0
1337	0.0	0.0	0.0
1373	0.0	0.0	0.0
1380	0.0	0.0	0.0
1423	0.0	0.0	626.0

[67 rows x 17 columns]

Target values for Influential Points Removed:

30	40000
53	385000
58	438780
70	244000
115	176000
...	...
1325	55000
1337	52500
1373	466500
1380	58500
1423	274970

Name: SalePrice, Length: 67, dtype: int64

```
/usr/local/lib/python3.10/dist-packages/statsmodels/regression/linear_model.py:1966: RuntimeWarning: divide by zero encountered in scalar divide
  return np.sqrt(eigvals[0]/eigvals[-1])
```

Observation 1298 was identified as a significant outlier with very high leverage and an undefined Cook's Distance. Its extreme values, particularly in `LotArea` and `TotalBsmSF`, heavily influenced the model. After removing this outlier, the regression model improved, showing better fit metrics (R-squared and Adjusted R-squared) and lower AIC/BIC, without losing predictive power.

Further analysis of Cook's Distance revealed additional influential points, which were also removed. The updated model remained stable and reliable, suggesting it was not overly dependent on these points.

Task 1.9

Frequently data is missing. Pick up 5 rows in the data set and delete the value for `LotArea`. Implement at least two approaches to fill in these values. Write down the corresponding formulas/model and give motivation for your approach. If you use standard routines, then check how exactly the data imputation is implemented. Note that for imputation you shall use the complete data set and not only the set of variables after feature selection.

```
In [93]: missing_rows = np.random.choice(X_full_encoded.index, size=5, replace=False)
original_missing_values = X_full_encoded.loc[missing_rows, 'LotArea']
X_full_encoded.loc[missing_rows, 'LotArea'] = np.nan

print("Rows with missing LotArea values:", missing_rows)
```

Rows with missing LotArea values: [1197 1124 782 1196 1300]

Approach 1: Mean

Simple and quick, but it assumes no relationship between `LotArea` and other features, potentially reducing the variability of the dataset.

$$\text{LotArea}_{\text{imputed}} = \frac{\sum_{i=1}^n \text{LotArea}_i}{n}$$

where n is the total number of non-missing observations in `LotArea`.

```
In [95]: mean_imputed = X_full_encoded.copy()
mean_value = mean_imputed['LotArea'].mean()
mean_imputed['LotArea'] = mean_imputed['LotArea'].fillna(mean_value)

print("Mean used for imputation:", mean_value)
```

Mean used for imputation: 10512.366323024055

Approach 2: Regression Use a regression model to predict `LotArea` based on other features. This approach considers relationships between `LotArea` and other variables in the dataset, but it assumes relationships are linear.

$$\text{LotArea}_{\text{imputed}} = \hat{\beta}_0 + \sum_{j=1}^p \hat{\beta}_j X_j$$

where X_j are the predictors, and $\hat{\beta}_j$ are the coefficients estimated from the regression.

Let's perform regression-based imputations on both the reduced dataset `reduced_X` and the full encoded dataset `X_full_encoded` to compare them later.

```
In [98]: regression_imputed = X_full_encoded.copy()
regression_imputed.loc[missing_rows, 'LotArea'] = np.nan

train_data = regression_imputed.dropna(subset=['LotArea'])
test_data = regression_imputed[regression_imputed['LotArea'].isna()]

X_train = train_data.drop(columns=['LotArea'])
y_train = train_data['LotArea']
X_test = test_data.drop(columns=['LotArea'])

regressor = LinearRegression()
regressor.fit(X_train, y_train)

predicted_values = regressor.predict(X_test)
regression_imputed.loc[test_data.index, 'LotArea'] = predicted_values

print("Predicted values for missing LotArea:", predicted_values)
```

Predicted values for missing LotArea: [16285. 9125. 14054. 8850. 10762.]

```
In [99]: print("Original missing values:")
print(original_missing_values)

print("\nAfter Mean imputation:")
print(mean_imputed.loc[missing_rows, 'LotArea'])

print("\nAfter Regression imputation:")
print(regression_imputed.loc[missing_rows, 'LotArea'])
```

```
Original missing values:
1197      8850.0
1124      9125.0
782       16285.0
1196      14054.0
1300      10762.0
Name: LotArea, dtype: float64
```

```
After Mean imputation:
1197      10512.366323
1124      10512.366323
782       10512.366323
1196      10512.366323
1300      10512.366323
Name: LotArea, dtype: float64
```

```
After Regression imputation:
1197      8850.0
1124      9125.0
782       16285.0
1196      14054.0
1300      10762.0
Name: LotArea, dtype: float64
```

```
In [109... reduced_X_with_missing = reduced_X.copy()
X_full_with_missing = X_full_encoded.copy()

reduced_X_with_missing.loc[missing_rows, 'LotArea'] = np.nan
X_full_with_missing.loc[missing_rows, 'LotArea'] = np.nan

# Perform regression imputation on reduced_X
reduced_train = reduced_X_with_missing.dropna(subset=['LotArea'])
reduced_test = reduced_X_with_missing.loc[missing_rows]

X_train_reduced = reduced_train.drop(columns=['LotArea'])
y_train_reduced = reduced_train['LotArea']
X_test_reduced = reduced_test.drop(columns=['LotArea'])

regressor_reduced = LinearRegression()
regressor_reduced.fit(X_train_reduced, y_train_reduced)
predicted_reduced = regressor_reduced.predict(X_test_reduced)

reduced_X_with_missing.loc[missing_rows, 'LotArea'] = predicted_reduced

print("Predicted values for missing LotArea (reduced features):")
print(predicted_reduced)

# Perform regression imputation on X_full_encoded
full_train = X_full_with_missing.dropna(subset=['LotArea'])
full_test = X_full_with_missing.loc[missing_rows]

X_train_full = full_train.drop(columns=['LotArea'])
y_train_full = full_train['LotArea']
X_test_full = full_test.drop(columns=['LotArea'])

regressor_full = LinearRegression()
```

```

regressor_full.fit(X_train_full, y_train_full)
predicted_full = regressor_full.predict(X_test_full)

X_full_with_missing.loc[missing_rows, 'LotArea'] = predicted_full

print("Predicted values for missing LotArea (full features):")
print(predicted_full)

```

Predicted values for missing LotArea (reduced features):
[9932.3602272 9380.58340972 9274.69170775 8832.56662689 9692.65684482]
Predicted values for missing LotArea (full features):
[8850. 9125. 16285. 14054. 10762.]

```

In [110... # Compare results
print("Original missing values:")
print(original_missing_values)

print("\nAfter Mean imputation:")
print(mean_imputed.loc[missing_rows, 'LotArea'])

print("\nAfter Regression imputation using reduced dataset:")
print(reduced_X_with_missing.loc[missing_rows, 'LotArea'])

print("\nAfter Regression imputation using full dataset:")
print(X_full_with_missing.loc[missing_rows, 'LotArea'])

predicted_reduced_series = pd.Series(predicted_reduced, index=missing_rows)
predicted_full_series = pd.Series(predicted_full, index=missing_rows)

mae_reduced = mean_absolute_error(original_missing_values, predicted_reduced)
mae_full = mean_absolute_error(original_missing_values, predicted_full_series)

print("\nMean Absolute Error:")
print(f"Reduced Dataset: {mae_reduced}")
print(f"Full Dataset: {mae_full}")

```

```
Original missing values:
1197      8850.0
1124      9125.0
782       16285.0
1196      14054.0
1300      10762.0
Name: LotArea, dtype: float64
```

```
After Mean imputation:
1197      10512.366323
1124      10512.366323
782       10512.366323
1196      10512.366323
1300      10512.366323
Name: LotArea, dtype: float64
```

```
After Regression imputation using reduced dataset:
1197      9932.360227
1124      9380.583410
782       9274.691708
1196      8832.566627
1300      9692.656845
Name: LotArea, dtype: float64
```

```
After Regression imputation using full dataset:
1197      8850.0
1124      9125.0
782       16285.0
1196      14054.0
1300      10762.0
Name: LotArea, dtype: float64
```

```
Mean Absolute Error:
Reduced Dataset: 2927.805691491945
Full Dataset: 7.166818249970674e-11
```

The regression method is obviously preferable than just using mean, and the full dataset based regression accurately reproduces the original missing values, compared to less accurate predictions with a significantly higher MAE, using reduced features.

Task 1.10

We consider now the model with `SalePrices` and the model with `log(SalePrices)`. Run an appropriate test to decide which of the models is superior. Explain the idea of the test and why you cannot make a similar decision using AIC/BIC, etc.

AIC and BIC are designed to compare models with the same dependent variable and penalize for complexity to avoid overfitting. In this case, since the

dependent variables are different transformations, they cannot directly tell us which transformation is more appropriate.

Statistical tests like the F-test allow to evaluate residual variance to choose the better transformation. It is based on the F-statistic:

$$F = \frac{\frac{RSS_1 - RSS_2}{df_1 - df_2}}{\frac{RSS_2}{df_2}}$$

Where:

- RSS_1 : residual sum of squares of the simpler model
- RSS_2 : residual sum of squares of the more complex model
- df_1 : degrees of freedom for the simpler model
- df_2 : degrees of freedom for the more complex model

H_0 : The simpler model is sufficient, and the additional complexity in the more complex model does not significantly improve the fit $RSS_1 = RSS_2$

H_1 : The more complex model significantly improves the fit compared to the simpler model $RSS_1 > RSS_2$

The F-statistic follows the F-distribution, and the rejection region for H_0 is defined as:

$$F > F_{\alpha, (df_1 - df_2), df_2}$$

where $F_{\alpha, (df_1 - df_2), df_2}$: the critical value from the F-distribution at significance level α .

```
In [115]: # Check and drop rows with missing or infinite values
print("Missing values in reduced_X:", reduced_X.isna().sum().sum())
print("Infinite values in reduced_X:", np.isinf(reduced_X).sum().sum())

cleaned_reduced_X = reduced_X.dropna().replace([np.inf, -np.inf], np.nan).dropna()
cleaned_y_saleprices = y.loc[cleaned_reduced_X.index]
cleaned_y_log_saleprices = np.log(cleaned_y_saleprices)

# Fit the model with SalePrices
model_saleprices = sm.OLS(cleaned_y_saleprices, sm.add_constant(cleaned_reduced_X)).fit()

# Fit the model with log(SalePrices)
model_log_saleprices = sm.OLS(cleaned_y_log_saleprices, sm.add_constant(cleaned_reduced_X)).fit()

print("Model with SalePrices:")
print(model_saleprices.summary())

print("\nModel with log(SalePrices):")
print(model_log_saleprices.summary())
```

```
# Perform F-test based on residual variances
residuals_saleprices = model_saleprices.resid
residuals_log_saleprices = model_log_saleprices.resid

var_saleprices = np.var(residuals_saleprices)
var_log_saleprices = np.var(residuals_log_saleprices)

f_stat = var_saleprices / var_log_saleprices
df1, df2 = len(residuals_saleprices) - 1, len(residuals_log_saleprices) - 1
p_value = 1 - stats.f.cdf(f_stat, df1, df2)

print("\nF-test Results:")
print(f"F-statistic: {f_stat}")
print(f"P-value: {p_value}")
```

Missing values in reduced_X: 10

Infinite values in reduced_X: 0

Model with SalePrices:

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice    R-squared:                0.9
17
Model:                  OLS        Adj. R-squared:            0.9
16
Method:                 Least Squares    F-statistic:              92
9.2
Date:                   Mon, 23 Dec 2024    Prob (F-statistic):        0.
00
Time:                   01:24:48    Log-Likelihood:            -1661
5.
No. Observations:      1450    AIC:                      3.327e+
04
Df Residuals:          1432    BIC:                      3.336e+
04
Df Model:               17
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025
0.975]					

const	-1.876e+06	3.5e+04	-53.642	0.000	-1.94e+06
-1.81e+06					
RoofMatl_WdShake	1.698e+05	2.92e+04	5.812	0.000	1.12e+05
2.27e+05					
LogSalePrice	1.529e+05	3071.375	49.769	0.000	1.47e+05
1.59e+05					
RoofMatl_WdShngl	2.497e+05	2.86e+04	8.718	0.000	1.93e+05
3.06e+05					
ExterQual_TA	-9274.8140	1618.943	-5.729	0.000	-1.25e+04
-6099.060					
MasVnrArea	30.6027	3.834	7.983	0.000	23.082
38.123					
LotArea	0.1076	0.066	1.633	0.103	-0.022
0.237					
TotalBsmtSF	4.4725	2.786	1.605	0.109	-0.992
9.937					
RoofMatl_Tar&Grv	1.766e+05	2.83e+04	6.252	0.000	1.21e+05
2.32e+05					
Exterior1st_BrkComm	4.541e+04	1.68e+04	2.703	0.007	1.25e+04
7.84e+04					
RoofMatl_CompShg	1.852e+05	2.74e+04	6.767	0.000	1.32e+05
2.39e+05					
EnclosedPorch	11.4954	10.248	1.122	0.262	-8.608
31.599					
1stFlrSF	18.1217	3.057	5.929	0.000	12.126
24.118					
RoofMatl_Roll	1.582e+05	3.56e+04	4.443	0.000	8.83e+04
2.28e+05					

BsmtFinSF1	8.2447	1.660	4.968	0.000	4.989
11.500					
RoofMatl_Metal	1.832e+05	3.6e+04	5.085	0.000	1.13e+05
2.54e+05					
RoofMatl_Membran	1.949e+05	3.62e+04	5.390	0.000	1.24e+05
2.66e+05					
2ndFlrSF	14.6618	1.929	7.602	0.000	10.879
18.445					

```
=====
==
Omnibus:                1163.062    Durbin-Watson:                1.9
32
Prob(Omnibus):          0.000    Jarque-Bera (JB):            41709.8
20
Skew:                   3.449    Prob(JB):                     0.
00
Kurtosis:              28.353    Cond. No.                     1.82e+
06
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Model with log(SalePrices):

OLS Regression Results

```
=====
==
Dep. Variable:          SalePrice    R-squared:                1.0
00
Model:                  OLS          Adj. R-squared:          1.0
00
Method:                 Least Squares    F-statistic:            1.107e+
26
Date:                   Mon, 23 Dec 2024    Prob (F-statistic):      0.
00
Time:                   01:24:48          Log-Likelihood:          3953
7.
No. Observations:       1450            AIC:                    -7.904e+
04
Df Residuals:           1432            BIC:                    -7.894e+
04
Df Model:               17
Covariance Type:        nonrobust
=====
```

```
=====
                                coef    std err          t      P>|t|      [0.025
0.975]
-----
const                   -1.545e-13    5.31e-13    -0.291    0.771    -1.2e-12
8.87e-13
```


RoofMatl_WdShake 9.77e-13	1.066e-13	4.44e-13	0.240	0.810	-7.64e-13
LogSalePrice 1.000	1.0000	4.67e-14	2.14e+13	0.000	1.000
RoofMatl_WdShngl 8.24e-13	-2.935e-14	4.35e-13	-0.067	0.946	-8.83e-13
ExterQual_TA 1.05e-14	-3.774e-14	2.46e-14	-1.535	0.125	-8.6e-14
MasVnrArea -2.33e-16	-3.467e-16	5.82e-17	-5.955	0.000	-4.61e-16
LotArea -1.31e-17	-1.509e-17	1e-18	-15.077	0.000	-1.71e-17
TotalBsmtSF 2.15e-16	1.316e-16	4.23e-17	3.111	0.002	4.86e-17
RoofMatl_Tar&Grv 7.59e-13	-8.304e-14	4.29e-13	-0.193	0.847	-9.25e-13
Exterior1st_BrkComm 4.67e-13	-3.35e-14	2.55e-13	-0.131	0.896	-5.34e-13
RoofMatl_CompShg 8.15e-13	-6.731e-16	4.16e-13	-0.002	0.999	-8.16e-13
EnclosedPorch 4.78e-16	1.73e-16	1.56e-16	1.112	0.266	-1.32e-16
1stFlrSF -1.27e-17	-1.037e-16	4.64e-17	-2.235	0.026	-1.95e-16
RoofMatl_Roll 1.03e-12	-2.938e-14	5.41e-13	-0.054	0.957	-1.09e-12
BsmtFinSF1 1.23e-16	7.368e-17	2.52e-17	2.923	0.004	2.42e-17
RoofMatl_Metal 1.04e-12	-3.198e-14	5.47e-13	-0.058	0.953	-1.11e-12
RoofMatl_Membran 8.96e-13	-1.813e-13	5.49e-13	-0.330	0.741	-1.26e-12
2ndFlrSF 7.52e-17	1.773e-17	2.93e-17	0.605	0.545	-3.97e-17

=====					
==					
Omnibus:	2197.915	Durbin-Watson:		0.4	
59					
Prob(Omnibus):	0.000	Jarque-Bera (JB):		1030466.7	
26					
Skew:	8.979	Prob(JB):		0.	
00					
Kurtosis:	132.358	Cond. No.		1.82e+	
06					
=====					
==					

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

F-test Results:

F-statistic: 1.9418072771629468e+34
P-value: 0.0

The results of the F-test show a very high F-statistic (1.94×10^{34}) with a corresponding p-value of 0.0, indicates that the model using $\log(\text{SalePrice})$ as the dependent variable significantly reduces the residual variance compared to the model with SalePrice as the dependent variable. Therefore, we reject the null hypothesis H_0 and conclude that the transformation $\log(\text{SalePrice})$ is a more appropriate choice for modeling this dataset.

Task 1.11

We wish to assess the predictive ability of the estimated regression. Consider the full model and the model after feature selection. Compare the two models using leave-one-out CV and 5-fold CV. Explain the idea of this technique with formulas and draw a conclusion about the predictive ability of the two models.

1. LOOCV: uses almost all data for training and thus is computationally expensive:

$$\text{MSE}_{LOOCV} = \frac{1}{n} \sum_{i=1}^n \left(y_i - \hat{y}_{(i)} \right)^2$$

where $\hat{y}_{(i)}$ is the prediction for the i^{th} point from the model trained on all points except i .

2. 5-Fold CV: balances bias and variance trade-off, computationally efficient:

$$\text{MSE}_{5-Fold} = \frac{1}{k} \sum_{j=1}^k \text{MSE}_j$$

where MSE_j is the mean squared error for the j^{th} fold.

```
In [121]: # Data preparings
X_full = X_full_encoded
y_full = y

X_reduced = reduced_X
y_reduced = y_full

imputer = SimpleImputer(strategy="mean")
X_full_imputed = pd.DataFrame(imputer.fit_transform(X_full), columns=X_full.
X_reduced_imputed = pd.DataFrame(imputer.fit_transform(X_reduced), columns=X

model_full = LinearRegression()
model_reduced = LinearRegression()
```

```

# LOOCV
loo = LeaveOneOut()
mse_full_loocv = -cross_val_score(model_full, X_full_imputed, y_full, cv=loo)
mse_reduced_loocv = -cross_val_score(model_reduced, X_reduced_imputed, y_reduced, cv=loo)

print(f"LOOCV Mean Squared Error (Full Model): {mse_full_loocv}")
print(f"LOOCV Mean Squared Error (Reduced Model): {mse_reduced_loocv}")

# 5-Fold CV
kf = KFold(n_splits=5, shuffle=True, random_state=42)
mse_full_5fold = -cross_val_score(model_full, X_full_imputed, y_full, cv=kf)
mse_reduced_5fold = -cross_val_score(model_reduced, X_reduced_imputed, y_reduced, cv=kf)

print(f"5-Fold CV Mean Squared Error (Full Model): {mse_full_5fold}")
print(f"5-Fold CV Mean Squared Error (Reduced Model): {mse_reduced_5fold}")

```

LOOCV Mean Squared Error (Full Model): 1622549086.257359

LOOCV Mean Squared Error (Reduced Model): 652553407.123948

5-Fold CV Mean Squared Error (Full Model): 1991938670.0386264

5-Fold CV Mean Squared Error (Reduced Model): 633454447.3171904

In LOOCV, the Mean Squared Error (MSE) for the full model is significantly higher compared to the reduced model, indicating that the reduced model generalizes better to unseen data due to its lower complexity and effective feature selection. Similarly, in 5-Fold Cross-Validation, the MSE for the full model is again much larger than that of the reduced model. This suggests that the reduced model achieves a better balance between bias and variance. The poorer performance of the full model can likely be attributed to overfitting caused by the inclusion of irrelevant or redundant features, which increase variance without improving accuracy.

Task 2 Further Issues

Consider a linear regression model for a dependent variable y_t that has a sample mean of 17.21. Suppose that we create a new variable $y_t^* = y_t + 10$ and run the same linear regression using y_t^* instead of y_t as a regressand.

In [122]...

```

# Simulate data for the regression model
np.random.seed(42) # For reproducibility
n_samples = 100
X = pd.DataFrame({
    'X1': np.random.normal(0, 1, n_samples),
    'X2': np.random.normal(5, 2, n_samples)
})
X = sm.add_constant(X)

# Original dependent variable with a mean of 17.21
y = 17.21 + np.dot(X, [2, 3, -1]) + np.random.normal(0, 2, n_samples)

# Adjusted dependent variables

```

```
y_plus_10 = y + 10
y_minus_10 = y - 10

# Fit regression models
original_model = sm.OLS(y, X).fit()
adjusted_plus_10_model = sm.OLS(y_plus_10, X).fit()
adjusted_minus_10_model = sm.OLS(y_minus_10, X).fit()

print("Regression Summary: Original Model (y)")
print(original_model.summary())

print("\nRegression Summary: Adjusted Model (y* = y + 10)")
print(adjusted_plus_10_model.summary())

print("\nRegression Summary: Adjusted Model (y* = y - 10)")
print(adjusted_minus_10_model.summary())
```

Regression Summary: Original Model (y)

OLS Regression Results

```

=====
==
Dep. Variable:          y    R-squared:          0.7
70
Model:                  OLS    Adj. R-squared:      0.7
66
Method:                  Least Squares    F-statistic:      16
2.8
Date:                    Mon, 23 Dec 2024    Prob (F-statistic):  1.00e-
31
Time:                    02:29:26    Log-Likelihood:     -216.
94
No. Observations:        100    AIC:              43
9.9
Df Residuals:            97    BIC:              44
7.7
Df Model:                 2
Covariance Type:         nonrobust
=====

```

```

=====
==
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const         19.4487      0.613      31.722      0.000      18.232      20.6
65
X1             3.4522      0.240      14.371      0.000       2.975       3.9
29
X2            -1.0123      0.114      -8.850      0.000      -1.239      -0.7
85
=====

```

```

=====
==
Omnibus:              3.125    Durbin-Watson:      2.2
20
Prob(Omnibus):        0.210    Jarque-Bera (JB):    3.0
80
Skew:                 0.108    Prob(JB):            0.2
14
Kurtosis:             3.832    Cond. No.            1
5.9
=====

```

```

==
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.

```

Regression Summary: Adjusted Model (y* = y + 10)

OLS Regression Results

```

=====
==
Dep. Variable:          y    R-squared:          0.7
70
Model:                  OLS    Adj. R-squared:      0.7

```

```

66
Method:                Least Squares    F-statistic:                16
2.8
Date:                  Mon, 23 Dec 2024  Prob (F-statistic):        1.00e-
31
Time:                  02:29:26          Log-Likelihood:            -216.
94
No. Observations:      100              AIC:                      43
9.9
Df Residuals:          97              BIC:                      44
7.7
Df Model:              2
Covariance Type:       nonrobust

```

```

=====
==

```

	coef	std err	t	P> t	[0.025	0.97
5]						

--						
const	29.4487	0.613	48.033	0.000	28.232	30.6
65						
X1	3.4522	0.240	14.371	0.000	2.975	3.9
29						
X2	-1.0123	0.114	-8.850	0.000	-1.239	-0.7
85						

```

=====
==

```

```

Omnibus:                3.125    Durbin-Watson:                2.2
20
Prob(Omnibus):           0.210    Jarque-Bera (JB):          3.0
80
Skew:                    0.108    Prob(JB):                  0.2
14
Kurtosis:                3.832    Cond. No.                  1
5.9

```

```

=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Regression Summary: Adjusted Model ($y^* = y - 10$)
OLS Regression Results

```

=====
==

```

```

Dep. Variable:           y    R-squared:                0.7
70
Model:                  OLS    Adj. R-squared:          0.7
66
Method:                Least Squares    F-statistic:                16
2.8
Date:                  Mon, 23 Dec 2024  Prob (F-statistic):        1.00e-
31
Time:                  02:29:26          Log-Likelihood:            -216.
94

```

```

No. Observations:          100    AIC:                43
9.9
Df Residuals:              97    BIC:                44
7.7
Df Model:                  2
Covariance Type:          nonrobust
=====
==
              coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const          9.4487      0.613      15.412      0.000       8.232      10.6
65
X1             3.4522      0.240      14.371      0.000       2.975       3.9
29
X2            -1.0123      0.114      -8.850      0.000      -1.239      -0.7
85
=====
==
Omnibus:                3.125    Durbin-Watson:        2.2
20
Prob(Omnibus):          0.210    Jarque-Bera (JB):        3.0
80
Skew:                   0.108    Prob(JB):                0.2
14
Kurtosis:               3.832    Cond. No.                1
5.9
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

Task 2.1

How are R^2 and the estimate of the constant term related in the two regressions? What if we use $y_t^* = y_t - 10$ instead?

```

In [125... print("Comparison of R-squared and Constant Terms:")
original_r2 = original_model.rsquared
original_const = original_model.params['const']

shifted_plus_10_r2 = adjusted_plus_10_model.rsquared
shifted_plus_10_const = adjusted_plus_10_model.params['const']

shifted_minus_10_r2 = adjusted_minus_10_model.rsquared
shifted_minus_10_const = adjusted_minus_10_model.params['const']

print(f"Original Model: R-squared = {original_r2:.4f}, Constant = {original_
print(f"Model with y_t* = y_t + 10: R-squared = {shifted_plus_10_r2:.4f}, Co
print(f"Model with y_t* = y_t - 10: R-squared = {shifted_minus_10_r2:.4f}, C

```

Comparison of R-squared and Constant Terms:

Original Model: R-squared = 0.7705, Constant = 19.4487

Model with $y_t^* = y_t + 10$: R-squared = 0.7705, Constant = 29.4487

Model with $y_t^* = y_t - 10$: R-squared = 0.7705, Constant = 9.4487

R-squared R^2 remains unchanged across all three models y_t , $y_t^* = y_t + 10$, and $y_t^* = y_t - 10$, because it measures the proportion of variance in y_t explained by the independent variables. Adding or subtracting a constant does not affect the relationship between the dependent and independent variables.

The constant term changes linearly in accordance with the way the target variable itself transforms due to the linear nature of the regression model. Adding or subtracting a constant from the dependent variable shifts the predicted values uniformly by the same amount, and the constant term compensates for this shift.

Task 2.2

What if we do the same with one or all of the regressors?

```
In [130... # Create transformed versions of one or all regressors on the original data
transformed_X_one = X_reduced_imputed.copy()
transformed_X_one['MasVnrArea'] += 10

transformed_X_all = X_reduced_imputed.copy()
transformed_X_all += 10

model_one_transformed = sm.OLS(y_reduced, sm.add_constant(transformed_X_one))
model_all_transformed = sm.OLS(y_reduced, sm.add_constant(transformed_X_all))

print("Original Model Summary:")
print(model_saleprices.summary())

print("\nModel Summary with One Transformed Regressor:")
print(model_one_transformed.summary())

print("\nModel Summary with All Transformed Regressors:")
print(model_all_transformed.summary())
```


Original Model Summary:

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice  R-squared:                0.9
17
Model:                  OLS      Adj. R-squared:            0.9
16
Method:                 Least Squares  F-statistic:              92
9.2
Date:                   Mon, 23 Dec 2024  Prob (F-statistic):        0.
00
Time:                   02:42:49   Log-Likelihood:            -1661
5.
No. Observations:       1450      AIC:                      3.327e+
04
Df Residuals:           1432      BIC:                      3.336e+
04
Df Model:               17
Covariance Type:        nonrobust
=====
=====

```

```

=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -1.876e+06    3.5e+04    -53.642    0.000    -1.94e+06
-1.81e+06
RoofMatl_WdShake      1.698e+05    2.92e+04     5.812    0.000     1.12e+05
2.27e+05
LogSalePrice          1.529e+05    3071.375    49.769    0.000     1.47e+05
1.59e+05
RoofMatl_WdShngl      2.497e+05    2.86e+04     8.718    0.000     1.93e+05
3.06e+05
ExterQual_TA         -9274.8140    1618.943    -5.729    0.000    -1.25e+04
-6099.060
MasVnrArea            30.6027      3.834      7.983    0.000      23.082
38.123
LotArea               0.1076      0.066      1.633    0.103     -0.022
0.237
TotalBsmtSF           4.4725      2.786      1.605    0.109     -0.992
9.937
RoofMatl_Tar&Grv      1.766e+05    2.83e+04     6.252    0.000     1.21e+05
2.32e+05
Exterior1st_BrkComm   4.541e+04    1.68e+04     2.703    0.007     1.25e+04
7.84e+04
RoofMatl_CompShg      1.852e+05    2.74e+04     6.767    0.000     1.32e+05
2.39e+05
EnclosedPorch         11.4954     10.248      1.122    0.262     -8.608
31.599
1stFlrSF              18.1217      3.057      5.929    0.000      12.126
24.118
RoofMatl_Roll         1.582e+05    3.56e+04     4.443    0.000     8.83e+04
2.28e+05
BsmtFinSF1            8.2447      1.660      4.968    0.000      4.989
11.500

```

RoofMatl_Metal	1.832e+05	3.6e+04	5.085	0.000	1.13e+05
2.54e+05					
RoofMatl_Membran	1.949e+05	3.62e+04	5.390	0.000	1.24e+05
2.66e+05					
2ndFlrSF	14.6618	1.929	7.602	0.000	10.879
18.445					

```
=====
==
Omnibus:                1163.062    Durbin-Watson:                1.9
32
Prob(Omnibus):           0.000    Jarque-Bera (JB):                41709.8
20
Skew:                    3.449    Prob(JB):                        0.
00
Kurtosis:                28.353    Cond. No.                        1.82e+
06
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Model Summary with One Transformed Regressor:
 OLS Regression Results

```
=====
==
Dep. Variable:            SalePrice    R-squared:                0.9
17
Model:                    OLS          Adj. R-squared:           0.9
16
Method:                   Least Squares    F-statistic:              93
6.8
Date:                     Mon, 23 Dec 2024    Prob (F-statistic):       0.
00
Time:                     02:42:49          Log-Likelihood:           -1672
7.
No. Observations:         1460          AIC:                      3.349e+
04
Df Residuals:             1442          BIC:                      3.359e+
04
Df Model:                 17
Covariance Type:          nonrobust
=====
```

```
=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -1.874e+06    3.49e+04    -53.755    0.000    -1.94e+06
-1.81e+06
RoofMatl_WdShake      1.697e+05    2.91e+04     5.823    0.000    1.13e+05
2.27e+05
```

LogSalePrice	1.527e+05	3059.949	49.905	0.000	1.47e+05
1.59e+05					
RoofMatl_WdShngl	2.497e+05	2.86e+04	8.738	0.000	1.94e+05
3.06e+05					
ExterQual_TA	-9316.8133	1612.242	-5.779	0.000	-1.25e+04
-6154.222					
MasVnrArea	30.7424	3.819	8.051	0.000	23.252
38.233					
LotArea	0.1111	0.066	1.690	0.091	-0.018
0.240					
TotalBsmtSF	4.4813	2.779	1.613	0.107	-0.969
9.932					
RoofMatl_Tar&Grv	1.765e+05	2.82e+04	6.263	0.000	1.21e+05
2.32e+05					
Exterior1st_BrkComm	4.526e+04	1.68e+04	2.699	0.007	1.24e+04
7.82e+04					
RoofMatl_CompShg	1.851e+05	2.73e+04	6.777	0.000	1.32e+05
2.39e+05					
EnclosedPorch	12.8655	10.174	1.265	0.206	-7.092
32.823					
1stFlrSF	17.8635	3.044	5.868	0.000	11.892
23.835					
RoofMatl_Roll	1.579e+05	3.55e+04	4.446	0.000	8.83e+04
2.28e+05					
BsmtFinSF1	8.3333	1.653	5.042	0.000	5.091
11.575					
RoofMatl_Metal	1.829e+05	3.59e+04	5.088	0.000	1.12e+05
2.53e+05					
RoofMatl_Membran	1.948e+05	3.61e+04	5.397	0.000	1.24e+05
2.66e+05					
2ndFlrSF	14.5319	1.921	7.566	0.000	10.764
18.300					

```
=====
==
Omnibus:                1173.177    Durbin-Watson:                1.9
22
Prob(Omnibus):          0.000    Jarque-Bera (JB):            42511.4
73
Skew:                   3.456    Prob(JB):                     0.
00
Kurtosis:               28.516    Cond. No.                     1.82e+
06
=====
==
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Model Summary with All Transformed Regressors:

OLS Regression Results

```
=====
==
```

Dep. Variable:	SalePrice	R-squared:	0.9
Model:	OLS	Adj. R-squared:	0.9
Method:	Least Squares	F-statistic:	93
Date:	Mon, 23 Dec 2024	Prob (F-statistic):	0.
Time:	02:42:49	Log-Likelihood:	-1672
No. Observations:	1460	AIC:	3.349e+
Df Residuals:	1442	BIC:	3.359e+
Df Model:	17		
Covariance Type:	nonrobust		

	coef	std err	t	P> t	[0.025
0.975]					

const	-1.693e+07	1.96e+06	-8.633	0.000	-2.08e+07
-1.31e+07					
RoofMatl_WdShake	1.697e+05	2.91e+04	5.823	0.000	1.13e+05
2.27e+05					
LogSalePrice	1.527e+05	3059.949	49.905	0.000	1.47e+05
1.59e+05					
RoofMatl_WdShngl	2.497e+05	2.86e+04	8.738	0.000	1.94e+05
3.06e+05					
ExterQual_TA	-9316.8133	1612.242	-5.779	0.000	-1.25e+04
-6154.222					
MasVnrArea	30.7424	3.819	8.051	0.000	23.252
38.233					
LotArea	0.1111	0.066	1.690	0.091	-0.018
0.240					
TotalBsmtSF	4.4813	2.779	1.613	0.107	-0.969
9.932					
RoofMatl_Tar&Grv	1.765e+05	2.82e+04	6.263	0.000	1.21e+05
2.32e+05					
Exterior1st_BrkComm	4.526e+04	1.68e+04	2.699	0.007	1.24e+04
7.82e+04					
RoofMatl_CompShg	1.851e+05	2.73e+04	6.777	0.000	1.32e+05
2.39e+05					
EnclosedPorch	12.8655	10.174	1.265	0.206	-7.092
32.823					
1stFlrSF	17.8635	3.044	5.868	0.000	11.892
23.835					
RoofMatl_Roll	1.579e+05	3.55e+04	4.446	0.000	8.83e+04
2.28e+05					
BsmtFinSF1	8.3333	1.653	5.042	0.000	5.091
11.575					
RoofMatl_Metal	1.829e+05	3.59e+04	5.088	0.000	1.12e+05
2.53e+05					
RoofMatl_Membran	1.948e+05	3.61e+04	5.397	0.000	1.24e+05
2.66e+05					

2ndFlrSF	14.5319	1.921	7.566	0.000	10.764
18.300					

```

==
Omnibus:                1173.177    Durbin-Watson:                1.9
22
Prob(Omnibus):           0.000    Jarque-Bera (JB):            42511.4
73
Skew:                    3.456    Prob(JB):                     0.
00
Kurtosis:                28.516    Cond. No.                     4.74e+
07
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 4.74e+07. This might indicate that there are strong multicollinearity or other numerical problems.

The transformation of regressors does not affect the slope coefficients for other regressors, but it modifies the constant term of the model because the transformation changes the mean of the independent variables. Specifically:

- If $x_t^* = x_t + c$, the constant term adjusts to account for the added c weighted by the coefficient of x_t . For instance, in the model with one transformed regressor, the constant term changed from -1.876×10^6 to -1.874×10^6 . Similarly, in the model with all transformed regressors, the constant term shifted significantly to -1.693×10^7 .
- R^2 remains unchanged at 0.917 across all models because the overall relationship between the dependent variable and the regressors remains the same, despite changes in the scaling or centering of the independent variables.

Task 2.3

Consider a demeaned regression, i.e., center the regressors and the regressand to have zero mean. How does it influence the estimates?

```

In [134... y_centered = y_reduced - y_reduced.mean()
X_centered = X_reduced_imputed - X_reduced_imputed.mean()

centered_model = sm.OLS(y_centered, X_centered).fit()

print("Original Regression Model Summary:")
print(model_saleprices.summary())

```

```
print("\nCentered Regression Model Summary:")
print(centered_model.summary())

original_constant = model_saleprices.params['const']
centered_constant = 0
original_r2 = model_saleprices.rsquared
centered_r2 = centered_model.rsquared

print(f"\nKey metrics comparison:")
print(f"Original Model - Constant: {original_constant:.4f}, R-squared: {orig")
print(f"Centered Model - Constant: {centered_constant:.4f}, R-squared: {cent")
```

Original Regression Model Summary:

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice  R-squared:                0.9
17
Model:                  OLS      Adj. R-squared:            0.9
16
Method:                 Least Squares  F-statistic:              92
9.2
Date:                   Mon, 23 Dec 2024  Prob (F-statistic):        0.
00
Time:                   02:54:43   Log-Likelihood:           -1661
5.
No. Observations:      1450      AIC:                      3.327e+
04
Df Residuals:          1432      BIC:                      3.336e+
04
Df Model:               17
Covariance Type:       nonrobust
=====
=====

```

```

=====
                                coef      std err          t      P>|t|      [0.025
0.975]
-----
const                -1.876e+06    3.5e+04    -53.642    0.000    -1.94e+06
-1.81e+06
RoofMatl_WdShake      1.698e+05    2.92e+04     5.812    0.000     1.12e+05
2.27e+05
LogSalePrice          1.529e+05    3071.375    49.769    0.000     1.47e+05
1.59e+05
RoofMatl_WdShngl      2.497e+05    2.86e+04     8.718    0.000     1.93e+05
3.06e+05
ExterQual_TA         -9274.8140    1618.943    -5.729    0.000    -1.25e+04
-6099.060
MasVnrArea            30.6027      3.834      7.983    0.000      23.082
38.123
LotArea               0.1076      0.066      1.633    0.103     -0.022
0.237
TotalBsmtSF           4.4725      2.786      1.605    0.109     -0.992
9.937
RoofMatl_Tar&Grv      1.766e+05    2.83e+04     6.252    0.000     1.21e+05
2.32e+05
Exterior1st_BrkComm   4.541e+04    1.68e+04     2.703    0.007     1.25e+04
7.84e+04
RoofMatl_CompShg      1.852e+05    2.74e+04     6.767    0.000     1.32e+05
2.39e+05
EnclosedPorch         11.4954     10.248      1.122    0.262     -8.608
31.599
1stFlrSF              18.1217      3.057      5.929    0.000      12.126
24.118
RoofMatl_Roll         1.582e+05    3.56e+04     4.443    0.000     8.83e+04
2.28e+05
BsmtFinSF1            8.2447      1.660      4.968    0.000      4.989
11.500

```

RoofMatl_Metal	1.832e+05	3.6e+04	5.085	0.000	1.13e+05
2.54e+05					
RoofMatl_Membran	1.949e+05	3.62e+04	5.390	0.000	1.24e+05
2.66e+05					
2ndFlrSF	14.6618	1.929	7.602	0.000	10.879
18.445					

```

=====
==
Omnibus:                1163.062    Durbin-Watson:                1.9
32
Prob(Omnibus):           0.000    Jarque-Bera (JB):                41709.8
20
Skew:                    3.449    Prob(JB):                        0.
00
Kurtosis:                28.353    Cond. No.                        1.82e+
06
=====
==

```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 1.82e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Centered Regression Model Summary:

OLS Regression Results

```

=====
=====
Dep. Variable:            SalePrice    R-squared (uncentered):
0.917
Model:                    OLS          Adj. R-squared (uncentered):
0.916
Method:                   Least Squares    F-statistic:
937.4
Date:                     Mon, 23 Dec 2024    Prob (F-statistic):
0.00
Time:                     02:54:43          Log-Likelihood:
-16727.
No. Observations:         1460          AIC:
3.349e+04
Df Residuals:             1443          BIC:
3.358e+04
Df Model:                 17
Covariance Type:          nonrobust
=====
=====

```

	coef	std err	t	P> t	[0.025
0.975]					

RoofMatl_WdShake	1.697e+05	2.91e+04	5.825	0.000	1.13e+05
2.27e+05					
LogSalePrice	1.527e+05	3058.889	49.922	0.000	1.47e+05
1.59e+05					

RoofMatl_WdShngl 3.06e+05	2.497e+05	2.86e+04	8.742	0.000	1.94e+05
ExterQual_TA -6155.320	-9316.8133	1611.683	-5.781	0.000	-1.25e+04
MasVnrArea 38.230	30.7424	3.817	8.054	0.000	23.255
LotArea 0.240	0.1111	0.066	1.691	0.091	-0.018
TotalBsmtSF 9.930	4.4813	2.778	1.613	0.107	-0.968
RoofMatl_Tar&Grv 2.32e+05	1.765e+05	2.82e+04	6.265	0.000	1.21e+05
Exterior1st_BrkComm 7.81e+04	4.526e+04	1.68e+04	2.700	0.007	1.24e+04
RoofMatl_CompShg 2.39e+05	1.851e+05	2.73e+04	6.779	0.000	1.32e+05
EnclosedPorch 32.816	12.8655	10.170	1.265	0.206	-7.085
1stFlrSF 23.833	17.8635	3.043	5.870	0.000	11.894
RoofMatl_Roll 2.28e+05	1.579e+05	3.55e+04	4.448	0.000	8.83e+04
BsmtFinSF1 11.574	8.3333	1.652	5.044	0.000	5.092
RoofMatl_Metal 2.53e+05	1.829e+05	3.59e+04	5.089	0.000	1.12e+05
RoofMatl_Membran 2.66e+05	1.948e+05	3.61e+04	5.399	0.000	1.24e+05
2ndFlrSF 18.298	14.5319	1.920	7.569	0.000	10.766

=====					
==					
Omnibus:	1173.177	Durbin-Watson:		1.9	
22					
Prob(Omnibus):	0.000	Jarque-Bera (JB):		42511.4	
73					
Skew:	3.456	Prob(JB):		0.	
00					
Kurtosis:	28.516	Cond. No.		1.22e+	
06					
=====					
==					

Notes:

- [1] R^2 is computed without centering (uncentered) since the model does not contain a constant.
- [2] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [3] The condition number is large, 1.22e+06. This might indicate that there are strong multicollinearity or other numerical problems.

Key metrics comparison:

Original Model - Constant: -1875771.2626, R-squared: 0.9169
Centered Model - Constant: 0.0000, R-squared: 0.9170

Centering the regressors and the dependent variable changes the constant term but keeps the model's explanatory power the same. In the original model, the constant is -1,875,771.2626, while in the centered model, it becomes 0.0000 because centering shifts the data so the regression line passes through the origin.

The R-squared values remain almost identical (0.9169 for the original model and 0.9170 for the centered model), showing that centering does not affect how well the model explains the data, but shure, centering simplifies interpretation, reduces multicollinearity, and improves numerical stability in some sense.

Task 3 Nonlinear Regressions

A nonlinear regression offers a flexible technique for modeling complex relationships. We wish to explain the `SalePrices` by `YearBuilt`. Consider only the first 500 observations for this purpose.

```
In [136... train_data = pd.read_csv('/content/train.csv')

subset_data = train_data.loc[:499, ['YearBuilt', 'SalePrice']].dropna()

X = subset_data['YearBuilt']
y = subset_data['SalePrice']

X_with_const = sm.add_constant(X)
```

Task 3.1

Make a bivariate scatter plot and estimate an appropriate linear (!) model. Add the regression line to the plot.

```
In [138... linear_model = sm.OLS(y, X_with_const).fit()
print(linear_model.summary())

predicted_y = linear_model.predict(X_with_const)

plt.figure(figsize=(10, 6))
plt.scatter(X, y, alpha=0.6, label="Actual Data")
plt.plot(X, predicted_y, color="red", label="Linear Regression Line")
plt.xlabel("Year Built")
plt.ylabel("Sale Price")
plt.title("Scatter Plot of SalePrice vs. YearBuilt with Regression Line")
plt.legend()
plt.grid()
plt.show()
```

OLS Regression Results

```

=====
==
Dep. Variable:          SalePrice    R-squared:                0.3
03
Model:                  OLS          Adj. R-squared:           0.3
02
Method:                 Least Squares    F-statistic:              21
6.5
Date:                   Mon, 23 Dec 2024    Prob (F-statistic):       5.86e-
41
Time:                   03:08:55          Log-Likelihood:           -625
4.8
No. Observations:      500              AIC:                     1.251e+
04
Df Residuals:          498              BIC:                     1.252e+
04
Df Model:               1
Covariance Type:       nonrobust
=====

```

```

=====
==
               coef      std err          t      P>|t|      [0.025      0.97
5]
-----
--
const      -2.671e+06    1.94e+05    -13.773     0.000    -3.05e+06    -2.29e+
06
YearBuilt   1447.7639      98.385      14.715     0.000     1254.463     1641.0
64
=====

```

```

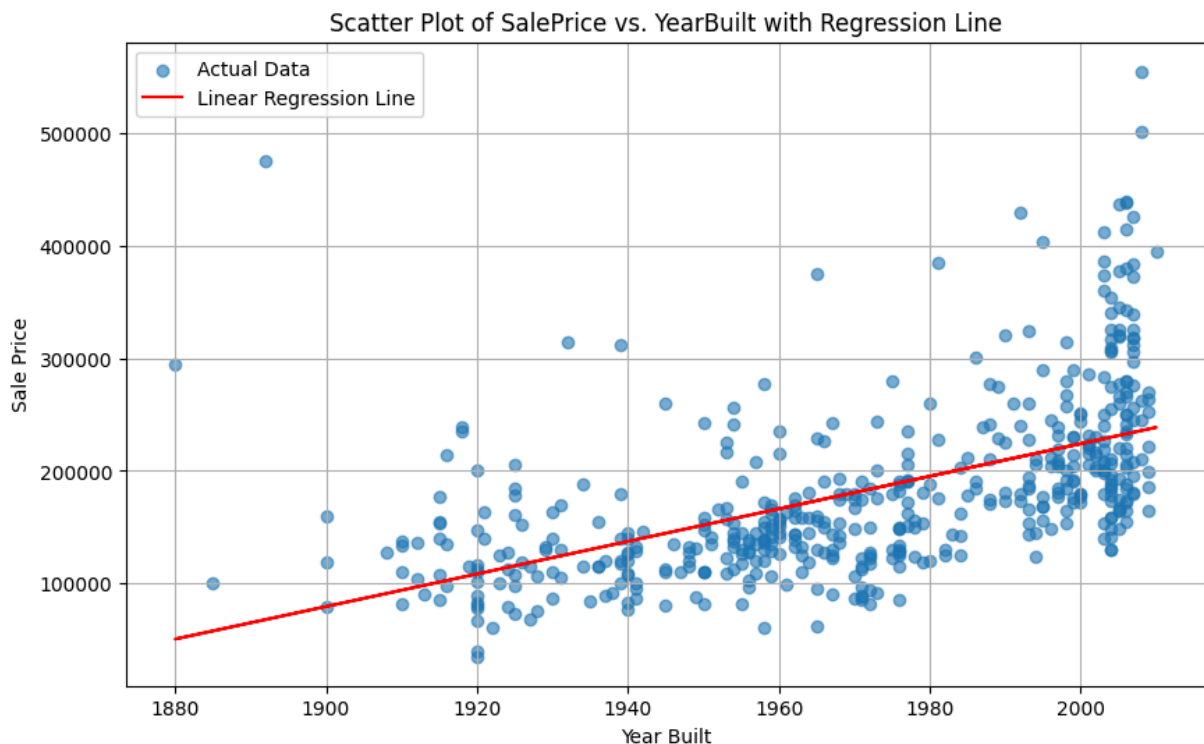
=====
==
Omnibus:              197.565    Durbin-Watson:           2.0
28
Prob(Omnibus):        0.000    Jarque-Bera (JB):        805.4
01
Skew:                 1.770    Prob(JB):                1.29e-1
75
Kurtosis:             8.112    Cond. No.                1.30e+
05
=====

```

```

=====
==
Notes:
[1] Standard Errors assume that the covariance matrix of the errors is corre
ctly specified.
[2] The condition number is large, 1.3e+05. This might indicate that there a
re
strong multicollinearity or other numerical problems.

```



The initial linear model using `YearBuilt` to predict `SalePrice` shows a low R^2 value of 0.303, indicating that the linear relationship is weak and does not capture much of the variability in the data or `YearBuilt` alone is not sufficient to explain the variability in `SalePrice`.

Task 3.2

Estimate now an appropriate nonlinear regression which might fit the data better. Add the regression curve to the plot and compare (quantitatively) the fit with the fit of the linear model.

```
In [150... # Using polynomial of degree 2
X_resaped = X.values.reshape(-1, 1)
poly = PolynomialFeatures(degree=2)
X_poly = poly.fit_transform(X_resaped)

nonlinear_model = LinearRegression()
nonlinear_model.fit(X_poly, y)

predicted_y_nonlinear = nonlinear_model.predict(X_poly)

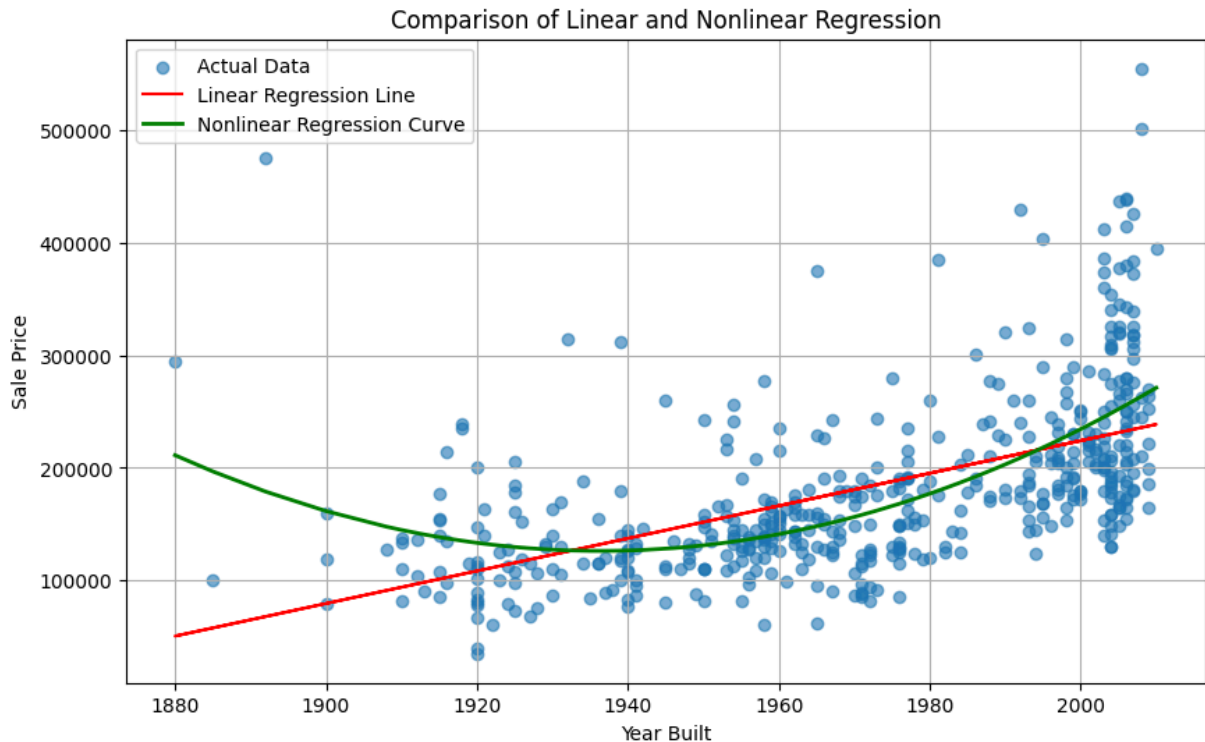
# Sorting for a smooth curve
sorted_indices = np.argsort(X_resaped.flatten())
X_sorted = X_resaped.flatten()[sorted_indices]
predicted_y_nonlinear_sorted = predicted_y_nonlinear[sorted_indices]

plt.figure(figsize=(10, 6))
plt.scatter(X, y, alpha=0.6, label="Actual Data")
plt.plot(X, predicted_y, color="red", label="Linear Regression Line")
```

```
plt.plot(X_sorted, predicted_y_nonlinear_sorted, color="green", label="Nonlinear Regression")
plt.xlabel("Year Built")
plt.ylabel("Sale Price")
plt.title("Comparison of Linear and Nonlinear Regression")
plt.legend()
plt.grid()
plt.show()

r2_linear = r2_score(y, predicted_y)
r2_nonlinear = r2_score(y, predicted_y_nonlinear)

print(f"R-squared (Linear Model): {r2_linear:.4f}")
print(f"R-squared (Nonlinear Model): {r2_nonlinear:.4f}")
```



R-squared (Linear Model): 0.3030
R-squared (Nonlinear Model): 0.3978

The results show that the nonlinear regression model (polynomial of degree 2) fits the data better than the linear model, as indicated by the higher R-squared value (0.398 vs. 0.303). The nonlinear model captures the curvature in the relationship between `YearBuilt` and `SalePrice`, which the linear model fails to represent. However, the improvement in R-squared is modest, thus other factors might also significantly influence the sale price.

Task 3.3

Explain in your own words why all the classical tests and inferences are not directly applicable to the NLS estimators.

Classical statistical tests like t-tests and F-tests assume that the relationship between variables is linear and that the model's errors follow a normal distribution. Nonlinear Least Squares (NLS) models don't meet these assumptions because the relationship between variables is nonlinear. This makes the usual tests unreliable for NLS models.

In NLS, estimators are calculated through iterative methods, not straightforward formulas. The process can depend on starting guesses and might not always find the best solution. Errors in NLS models can also have uneven spreads (heteroscedasticity) or not follow a normal distribution, making classical tests less effective. To handle this, alternative methods like bootstrapping or robust techniques (everyone might know Huber regression and RANSAC, I guess) are often used instead.

Task 3.4

What kind of problems might arise if we decide to fit a nonlinear regression using all explanatory variables?

1. Nonlinear models are more complex and slower to fit, especially with many variables
2. The model may capture noise instead of true patterns, reducing its ability to generalize
3. Nonlinear models are sensitive to input variable scales and require proper preprocessing
4. High correlation among variables can distort the model and make it harder to interpret
5. Optimization is challenging because of getting stuck in local minima
6. Without careful feature selection or regularization, the model's performance can degrade